The Beginner's Guide to Building AI Applications

LangChain · LangGraph · RAG · LlamaIndex

Pinecone · Weaviate · ChromaDB

Python · PyTorch · TensorFlow · Keras

Concepts, working examples, and hands-on projects

Written for absolute beginners — with interview preparation throughout

Table of Contents

1. How to Use This Guide

Who this is for

This guide assumes you know how to write basic Python — variables, functions, loops, lists, dictionaries — and nothing else. You do not need a machine learning background. You do not need to know what a neural network is. Every concept is built from scratch.

The guide is written so that you can read it front to back and never hit a term that was not explained earlier. If you skip around, use the Glossary at the end.

How to read it

There are three kinds of content, and you should treat each differently:

- **Concepts** — read these carefully. They are the part that gets tested in interviews. If you understand the concept, you can always look up the syntax.
- **Code examples** — type them out. Do not copy-paste. Typing forces you to notice details.
- **Projects** — build them. Nothing else creates the fluency that being able to say "yes, I built that" requires.

Throughout, you will see four kinds of callout boxes:

> ANALOGY
>
> Explains a hard idea by comparing it to something you already understand. Read these when a concept isn't landing.

> KEY IDEA
>
> The single most important sentence in a section. If you remember nothing else, remember these.

> COMMON MISTAKE
>
> Something beginners get wrong, or a trap in the library. Reading these will save you hours of debugging.

> INTERVIEW QUESTION
>
> **Q: What does this box do?**
>
> It gives you a question an interviewer is likely to ask, along with a good answer. Practise saying these out loud — knowing an answer and being able to *speak* it under pressure are different skills.

Setting up your environment

Everything in this guide runs on Python 3.10 or newer. Create a virtual environment first — this keeps the packages for this project separate from the rest of your machine, which prevents version conflicts.

```
# Create an isolated environment
python -m venv venv

# Activate it
source venv/bin/activate        # macOS / Linux
venv\Scripts\activate           # Windows

# Install everything this guide uses
pip install langchain langchain-openai langchain-community langgraph
pip install llama-index chromadb pinecone-client weaviate-client
pip install pypdf tiktoken sentence-transformers
pip install torch tensorflow scikit-learn pandas numpy matplotlib
```

You will also need an API key from a model provider. Store it in an environment variable — **never** paste a key directly into

your code, because the moment you push that file to GitHub, bots will find and drain your account within minutes.

```
# In your terminal
export OPENAI_API_KEY="sk-..."          # macOS / Linux
setx OPENAI_API_KEY "sk-..."            # Windows

# Or, better, in a .env file (add .env to .gitignore!)
# .env
OPENAI_API_KEY=sk-...

# And in Python:
from dotenv import load_dotenv
load_dotenv()
```

COMMON MISTAKE

Committing API keys to git. Add `.env` to your `.gitignore` on day one. If you ever do leak a key, revoke it immediately from the provider's dashboard — do not just delete the commit, because the key still exists in git history.

A note on cost

API calls cost money — usually fractions of a cent, but a runaway loop in an agent can burn through dollars fast. Three habits protect you:

- Set a **hard spending limit** in your provider's billing dashboard. Do this before you write any code.
- Use the **cheapest model** while developing (e.g. a "mini" or "haiku" tier). Switch to the expensive model only when the logic works.
- Always set a **recursion limit** on agents so a stuck loop terminates instead of running forever.

You can also run everything in this guide for free using local models via Ollama, at the cost of speed and quality. If cost is a concern, install Ollama, run `ollama pull llama3`, and substitute `ChatOllama(model="llama3")` wherever you see `ChatOpenAI(...)`.

2. Part I — The Foundations

Before touching any library, you need four ideas. Everything in this guide is built on them. Do not skip this chapter — most people who find LangChain confusing are actually confused about one of these four things.

2.1 What an LLM actually is

A Large Language Model is a function. It takes text in, and it produces text out. That is the whole thing.

```
output_text = llm(input_text)
```

More precisely, it predicts the next word (technically, the next *token*) over and over, feeding each prediction back in, until it decides to stop. That is genuinely all it does at inference time. It was trained by being shown enormous quantities of text and repeatedly asked "what comes next?" — and adjusting its billions of internal numbers to get better at that one question.

This simple mechanism produces surprisingly capable behaviour. But the mechanism also tells you exactly what an LLM *cannot* do, which is the part that matters for building applications:

An LLM has no...	Which means...
Memory	Each call is completely independent. It does not remember your previous message. If a chatbot seems to remember, it is because the entire conversation was silently re-sent with the new message.
Access to the outside world	It cannot browse the web, read your files, query your database, or check today's date. It only knows what was in its training data.
Knowledge of recent events	Its training data has a cutoff date. Anything after that date does not exist to it.
Ability to verify itself	When it doesn't know something, it does not say "I don't know." It produces the most statistically plausible-looking text. This is what people call **hallucination**.
Ability to do reliable arithmetic	It is predicting text, not calculating. It will confidently produce `17 × 24 = 418`. (It's 408.)

> KEY IDEA
>
> Every framework in this guide — LangChain, LangGraph, LlamaIndex, vector databases — exists to work around one of the five limitations in that table. When you understand what the limitation is, the tool's purpose becomes obvious.

2.2 Tokens, context windows, and temperature

Tokens

An LLM does not see letters or words. It sees **tokens** — chunks of text, roughly 3-4 characters each. The word "hamburger" might be three tokens: `ham`, `bur`, `ger`. Common words are usually one token. Rare words and non-English text use more.

Two practical consequences:

- **You pay per token.** Both for what you send (input tokens) and what you get back (output tokens). Output tokens usually cost 3-5× more than input tokens.
- **Rough rule:** 1 token ≈ 0.75 English words. So 1,000 tokens ≈ 750 words ≈ 1.5 pages.

```
import tiktoken

encoder = tiktoken.get_encoding("cl100k_base")
text = "LangChain makes building LLM apps easier."

tokens = encoder.encode(text)
print(f"Token count: {len(tokens)}")        # Token count: 8
print(f"Token IDs:   {tokens}")             # [27317, 26268, 3727, ...]

# Decode a single token to see what it actually is
print(encoder.decode([tokens[0]]))          # 'Lang'
```

The context window

The **context window** is the maximum number of tokens the model can consider at once — your prompt *and* its answer combined. If a model has a 128,000-token context window, and your prompt is 127,000 tokens, it has only 1,000 tokens left to answer with.

> ANALOGY
>
> Think of the context window as a whiteboard of a fixed size. Everything the model can "see" while thinking must fit on that whiteboard — the conversation history, any documents you pasted in, the system instructions, and the space for its own answer. When the board is full, something has to be erased.

This single constraint is why RAG exists (Part III). You cannot paste a 500-page manual into the prompt. So instead, you find the three most relevant paragraphs and paste only those.

Temperature

At each step, the model produces a probability distribution over all possible next tokens. **Temperature** controls how it picks from that distribution.

Temperature	Behaviour	Use for
`0.0`	Always picks the highest-probability token. Deterministic and repeatable.	Data extraction, classification, code generation, RAG answers, anything factual
`0.7`	Balanced. Some variation, still coherent.	Chatbots, general conversation
`1.0+`	Creative and unpredictable. Can go off the rails.	Brainstorming, fiction, marketing copy variations

> COMMON MISTAKE
>
> Leaving temperature at the default (often 0.7) for a RAG system, then wondering why the same question gives slightly different answers each time. **For anything factual, set** `temperature=0`**.** This is a very common interview follow-up: "your RAG system gives inconsistent answers — why?"

2.3 Embeddings — turning meaning into numbers

This is the single most important concept in this guide. If you understand embeddings, RAG and vector databases become trivial. If you don't, they will never make sense.

An **embedding** is a list of numbers that represents the *meaning* of a piece of text. A separate model — an **embedding model**, not the LLM — reads your text and outputs a fixed-length list of floats, typically 384, 768, or 1536 numbers long.

```
embed("The cat sat on the mat")
# → [0.021, -0.118, 0.334, ..., 0.087]   (1536 numbers)

embed("A feline rested on the rug")
# → [0.019, -0.121, 0.329, ..., 0.091]   (very similar numbers!)

embed("The stock market crashed")
# → [-0.412, 0.663, -0.087, ..., -0.223] (completely different numbers)
```

Notice what happened. The first two sentences share almost no words — "cat"/"feline", "mat"/"rug", "sat"/"rested" — yet their embeddings are nearly identical. The third sentence, about something entirely unrelated, has a wildly different embedding.

> KEY IDEA
>
> Embeddings convert **semantic similarity** (similar meaning) into **geometric proximity** (nearby points in space). Text that means the same thing lands in the same neighbourhood, even if it uses completely different words.

> ANALOGY
>
> Imagine a giant library where books are shelved not alphabetically, but by *topic* — and not just broad topic, but precise meaning. Every cookbook is in one corner. Within that corner, the Italian cookbooks cluster together. Within those, the pasta books are adjacent. You could walk to the pasta section without knowing a single book title. An embedding is the coordinate of a book on those shelves — except the shelves have 1,536 dimensions instead of 3.

Measuring similarity

Once text is a list of numbers, "how similar are these two texts?" becomes "how close are these two points?" The standard measure is **cosine similarity**, which measures the angle between two vectors:

- `1.0` — pointing in exactly the same direction (identical meaning)
- `0.0` — perpendicular (unrelated)
- `-1.0` — opposite directions (opposite meaning)

Cosine similarity ignores *magnitude* and only looks at *direction*, which is what you want: a long document and a short sentence about the same topic should be considered similar.

```
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Using a free, local embedding model — no API key needed
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')   # 384 dimensions

sentences = [
    "The cat sat on the mat",
    "A feline rested on the rug",
    "The stock market crashed today",
]
vectors = model.encode(sentences)

print(f"Vector length: {len(vectors[0])}")                        # 384
print(f"cat/feline:    {cosine_similarity(vectors[0], vectors[1]):.3f}")  # ~0.72
print(f"cat/stocks:    {cosine_similarity(vectors[0], vectors[2]):.3f}")  # ~0.05
```

Run this. Seeing 0.72 versus 0.05 with your own eyes is the moment embeddings click.

> COMMON MISTAKE
>
> Using a different embedding model for indexing than for querying. The vectors from two different models live in *completely different spaces* — comparing them produces meaningless similarity scores. **Whatever model you embedded your documents with, you must use the same model to embed your queries.** If you change embedding models, you must re-index everything.

INTERVIEW QUESTION

Q: What is an embedding, and why is it useful?

An embedding is a dense vector representation of text where semantic similarity corresponds to geometric proximity. It's useful because it lets you do semantic search — finding text that *means* the same thing as your query, rather than text that merely *contains the same keywords*. A keyword search for "car" misses a document about "automobiles"; a semantic search finds it.

2.4 The three ways to give an LLM knowledge

Your model doesn't know your company's internal policies. There are exactly three ways to fix that, and knowing when to use which is a guaranteed interview question.

Method	How it works	Cost	Use when
Prompting (in-context learning)	Paste the knowledge directly into the prompt.	Free (just tokens)	The knowledge is small — a few paragraphs, a style guide, 3-5 examples.
RAG (retrieval)	Store knowledge externally. At query time, fetch only the relevant bits and paste those into the prompt.	Low — embedding + storage	The knowledge is large, changes often, or you need citations. **This is the default answer for most business problems.**
Fine-tuning	Actually update the model's weights by training it on your data.	High — GPUs, data prep, time	You need to change the model's *behaviour, style, or format* — not add facts.

KEY IDEA

RAG teaches the model *what* to say. Fine-tuning teaches it *how* to say it. If you need the model to know your Q3 revenue figures, use RAG. If you need it to always respond in your company's specific tone and JSON schema, fine-tune. Most people who think they need fine-tuning actually need RAG.

INTERVIEW QUESTION

Q: When would you choose RAG over fine-tuning?

RAG when the knowledge is **factual, large, or changes frequently** — because updating a RAG system is just re-indexing a document, whereas updating a fine-tuned model means retraining. RAG also gives you **citations**, which you cannot get from a fine-tuned model, and that matters enormously for trust and compliance. Fine-tuning when you need to change **behaviour** — output format, tone, domain-specific reasoning style — or when you need to compress a very long system prompt into the weights to save latency and cost. They are not mutually exclusive; production systems often use both.

3. Part II — LangChain

3.1 The problem LangChain solves

Suppose you want to build a customer support bot. You need to: format a prompt with the customer's question, call a model, parse the reply into structured JSON, remember the conversation so far, and look up the customer's order in your database.

You could write all that yourself. Many people do. But you'd be writing the same boilerplate that thousands of other developers are writing — string templating, retry logic, JSON repair, conversation trimming, provider-specific API quirks.

LangChain is a library of standard, reusable components for building LLM applications, plus a way to snap those components together. That's it. It is not magic and it is not an AI. It is plumbing.

> ANALOGY
>
> LangChain is to LLM apps what jQuery was to the early web, or what Express is to Node servers. It doesn't do anything you couldn't do by hand — it just means you don't *have* to do it by hand, and everyone's code looks the same, so it's easier to read and hire for.

The package structure

LangChain was split into several packages. Knowing which is which prevents a lot of import confusion:

Package	Contains
`langchain-core`	The base abstractions: `Runnable`, message types, prompt templates. Almost no dependencies.
`langchain-openai`, `langchain-anthropic`, etc.	Provider-specific integrations. One package per provider.
`langchain-community`	Third-party integrations: document loaders, vector stores, tools.
`langchain`	The higher-level chains and agents built on top of core.
`langgraph`	Separate library for stateful, cyclical workflows. Covered in Part IV.

3.2 Component 1 — Models

LangChain wraps every provider behind one interface. This means you can swap OpenAI for Anthropic for a local Llama by changing a single line.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,          # deterministic — important for factual tasks
    max_tokens=500,         # cap the response length (cost control)
)

response = llm.invoke("Explain what an API is in one sentence.")
print(response.content)
# "An API is a set of rules that lets one piece of software
#  request services or data from another."
```

Message types

Chat models don't take a plain string — they take a *list of messages*, each with a role. There are three roles you must know:

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage(content="You are a terse assistant. Answer in under 10 words."),
    HumanMessage(content="What is Python?"),
    AIMessage(content="A high-level, general-purpose programming language."),
    HumanMessage(content="Who created it?"),
]

response = llm.invoke(messages)
print(response.content)   # "Guido van Rossum, in 1991."
```

Role	Purpose
`SystemMessage`	Instructions about *how* the model should behave. Persona, rules, output format. Sent once, at the top.
`HumanMessage`	What the user said.
`AIMessage`	What the model said previously. You include these to give it conversation history.
`ToolMessage`	The result of a tool call. Covered in section 3.6.

KEY IDEA

Notice what just happened in that code: the "conversation history" is **just a list you re-send every time**. The model has no memory of its own. Everything in LangChain called "memory" is ultimately a strategy for managing this list.

Streaming

Users hate waiting. Streaming shows tokens as they're generated, which makes the app *feel* instant even though total time is identical.

```
for chunk in llm.stream("Write a haiku about databases."):
    print(chunk.content, end="", flush=True)
```

3.3 Component 2 — Prompt Templates

A prompt template is a string with placeholders. You define it once and fill it in many times.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert {domain} tutor. Explain simply, for a beginner."),
    ("human", "Explain: {topic}"),
])

# .format_messages() fills the holes and returns the message list
messages = prompt.format_messages(domain="biology", topic="mitochondria")
print(messages)
# [SystemMessage('You are an expert biology tutor...'),
#  HumanMessage('Explain: mitochondria')]

response = llm.invoke(messages)
```

Few-shot prompting

The single highest-leverage prompting technique: instead of *describing* what you want, *show examples*. Models are far better at pattern-matching than at following abstract instructions.

```
from langchain_core.prompts import ChatPromptTemplate, FewShotChatMessagePromptTemplate

examples = [
    {"input": "I love this product!",        "output": "positive"},
    {"input": "Worst purchase of my life.",  "output": "negative"},
    {"input": "It arrived on Tuesday.",      "output": "neutral"},
]

example_prompt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}"),
])

few_shot = FewShotChatMessagePromptTemplate(
    example_prompt=example_prompt,
    examples=examples,
)

final_prompt = ChatPromptTemplate.from_messages([
    ("system", "Classify the sentiment. Reply with one word only."),
    few_shot,
    ("human", "{input}"),
])

chain = final_prompt | llm
print(chain.invoke({"input": "The packaging was damaged but it works."}).content)
# → "neutral"
```

COMMON MISTAKE

Using single curly braces in a prompt when you mean a literal brace — for example, asking for JSON output and writing `{"name": "..."}` in the template. LangChain will try to substitute a variable called `"name"` and crash. **Escape literal braces by doubling them:** `{{"name": "..."}}`.

3.4 Component 3 — Output Parsers

The model returns a string. Your application almost certainly wants a Python object. Output parsers bridge that gap — and, crucially, they also *tell the model what format to produce*.

The simplest parser

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()
result = chain.invoke({"domain": "physics", "topic": "gravity"})

print(type(result))   # <class 'str'>  — not an AIMessage object
# StrOutputParser just does: response.content
```

Structured output with Pydantic — the important one

This is the pattern you will use constantly in production. You define the shape of the data you want as a Python class, and the model is forced to produce data matching that shape.

```python
from pydantic import BaseModel, Field
from typing import List, Literal

class ResumeAnalysis(BaseModel):
    """Structured analysis of a candidate's resume."""
    name: str = Field(description="Candidate's full name")
    years_experience: int = Field(description="Total years of professional experience")
    top_skills: List[str] = Field(description="Up to 5 key technical skills")
    seniority: Literal["junior", "mid", "senior"] = Field(
        description="Assessed seniority level"
    )
    recommend: bool = Field(description="True if worth interviewing")

# .with_structured_output() forces the model to return this shape
structured_llm = llm.with_structured_output(ResumeAnalysis)

result = structured_llm.invoke("""
    Sarah Chen. 6 years building backend systems in Python and Go.
    Led a team of 4. Expert in Kubernetes, PostgreSQL, and gRPC.
""")

print(type(result))              # <class 'ResumeAnalysis'> — a real Python object!
print(result.name)               # "Sarah Chen"
print(result.years_experience)   # 6
print(result.seniority)          # "senior"
print(result.top_skills)         # ['Python', 'Go', 'Kubernetes', 'PostgreSQL', 'gRPC']
print(result.recommend)          # True
```

KEY IDEA

`with_structured_output()` is how you turn an unpredictable text generator into a reliable component of a software system. The docstrings and `Field(description=...)` are not decoration — they are sent to the model as instructions. Write them carefully.

COMMON MISTAKE

Asking the model for JSON in a plain prompt and then calling `json.loads()` on the result. It works 95% of the time, and fails on the 5% where the model wraps the JSON in a markdown code fence, or adds "Here's the JSON you asked for:" before it. Use `with_structured_output()` — it uses the provider's native function-calling API under the hood, which is far more reliable than hoping.

3.5 LCEL — the pipe operator and Runnables

This is the concept that unlocks LangChain, and it is **the single most likely LangChain interview question.**

Every component in LangChain — models, prompts, parsers, retrievers, and even functions you write yourself — implements a shared interface called `Runnable`. A `Runnable` promises four methods:

Method	Does
`.invoke(x)`	Run on one input, return one output.
`.batch([x, y, z])`	Run on many inputs, in parallel. Much faster than a loop.
`.stream(x)`	Run and yield the output piece by piece as it's produced.
`.ainvoke(x)`	The async version. Also `.abatch()`, `.astream()`.

Because every component speaks the same interface, you can pipe them together with `|`. The output of one becomes the input of the next — exactly like a Unix pipe.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "Write a one-sentence product description for: {product}"
)
llm    = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
parser = StrOutputParser()

# THE CHAIN. Read left to right: dict → messages → AIMessage → str
chain = prompt | llm | parser

# Run it on one input
print(chain.invoke({"product": "a mechanical keyboard"}))

# Run it on many inputs — in PARALLEL, not a loop
results = chain.batch([
    {"product": "a mechanical keyboard"},
    {"product": "noise-cancelling headphones"},
    {"product": "a standing desk"},
])
for r in results:
    print("-", r)

# Stream it
for token in chain.stream({"product": "an ergonomic mouse"}):
    print(token, end="", flush=True)
```

KEY IDEA — MEMORISE THIS FOR THE INTERVIEW

LCEL works because every component implements the `Runnable` interface. Since a chain built with `|` is *itself* a Runnable, chains can be nested inside other chains. And because the interface includes `.batch()`, `.stream()`, and async variants, **you get parallelism, streaming, and async for free on any chain you build** — you never write that code yourself.

Running things in parallel

`RunnableParallel` (or just a plain dict) runs multiple branches at once and collects results:

```
from langchain_core.runnables import RunnableParallel

summary_chain  = ChatPromptTemplate.from_template("Summarise in 10 words: {text}") | llm | parser
sentiment_chain = ChatPromptTemplate.from_template("Sentiment of this (one word): {text}") | llm | parser
keywords_chain  = ChatPromptTemplate.from_template("3 keywords, comma-separated: {text}") | llm | parser

# All three run SIMULTANEOUSLY — total time = slowest branch, not the sum
analyser = RunnableParallel(
    summary=summary_chain,
    sentiment=sentiment_chain,
    keywords=keywords_chain,
)

result = analyser.invoke({"text": "The new update fixed the crashes but the UI is now confusing."})
print(result)
# {'summary':   'Update fixed crashes but introduced confusing user interface.',
#  'sentiment': 'mixed',
#  'keywords':  'update, crashes, UI'}
```

Passing values through and injecting custom logic

```
from langchain_core.runnables import RunnablePassthrough, RunnableLambda

# RunnablePassthrough: forward the input unchanged
# RunnableLambda: wrap any plain Python function into a Runnable

def count_words(text: str) -> str:
    return f"{text}\n\n[Word count: {len(text.split())}]"

chain = (
    prompt
    | llm
    | parser
    | RunnableLambda(count_words)     # your own function, in the chain
)

print(chain.invoke({"product": "a coffee grinder"}))
```

The `RunnablePassthrough` pattern below is the exact structure of every RAG chain you will ever write — study it now, and Part III will be easy:

```
rag_chain = (
    {
        "context":  retriever,              # fetches docs from the question
        "question": RunnablePassthrough(),  # forwards the question unchanged
    }
    | prompt      # a template with {context} and {question} holes
    | llm
    | parser
)

rag_chain.invoke("What is the refund policy?")
```

Fallbacks and retries — production hygiene

```
# If the primary model fails (rate limit, outage), automatically try the backup
robust_llm = ChatOpenAI(model="gpt-4o").with_fallbacks([
    ChatOpenAI(model="gpt-4o-mini"),
])

# Retry with exponential backoff on transient failures
resilient = llm.with_retry(stop_after_attempt=3)
```

3.6 Component 4 — Tools and function calling

An LLM cannot check the weather. But it *can* tell you that it wants to. That is the entire trick behind tool calling.

> **KEY IDEA — HOW TOOL CALLING ACTUALLY WORKS**
>
> 1. You describe your tools to the model (name, description, parameters).
> 2. The model, instead of answering, returns a structured request: *"call `get_weather` with `{city: 'Lahore'}`"*.
> 3. **Your code** — not the model — actually runs that function.
> 4. You send the result back to the model.
> 5. The model uses the result to write a final answer.

The model never executes anything. It only *requests*. This is a critical security and mental-model point, and interviewers ask about it.

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a given city.

    Args:
        city: The name of the city, e.g. 'Lahore' or 'London'.
    """
    # In reality: call a weather API here
    return f"It is 34°C and sunny in {city}."

@tool
def calculator(expression: str) -> str:
    """Evaluate a mathematical expression. Use this for ALL arithmetic.

    Args:
        expression: A Python math expression, e.g. '17 * 24'.
    """
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"Error: {e}"

tools = [get_weather, calculator]

# Bind the tools to the model — this sends their schemas with every request
llm_with_tools = llm.bind_tools(tools)
```

COMMON MISTAKE

Writing a vague docstring. **The docstring is the only thing the model sees to decide whether to use your tool.** "Does math" is a bad description. "Evaluate a mathematical expression. Use this for ALL arithmetic — never calculate mentally" is a good one. If your agent isn't calling a tool it should be calling, the docstring is almost always the culprit.

Now watch the full loop, written by hand so you can see every step:

```
from langchain_core.messages import HumanMessage, ToolMessage

messages = [HumanMessage(content="What's the weather in Lahore, and what is 17 * 24?")]

# --- STEP 1: the model decides which tools to call ---
ai_msg = llm_with_tools.invoke(messages)
messages.append(ai_msg)

print(ai_msg.content)     # '' — it has NO text answer yet
print(ai_msg.tool_calls)  # [{'name': 'get_weather', 'args': {'city': 'Lahore'}, 'id': 'call_1'},
                          #  {'name': 'calculator',  'args': {'expression': '17*24'}, 'id': 'call_2'}]

# --- STEP 2: YOUR CODE runs the tools ---
tool_registry = {"get_weather": get_weather, "calculator": calculator}

for call in ai_msg.tool_calls:
    chosen_tool = tool_registry[call["name"]]
    output = chosen_tool.invoke(call["args"])
    # The tool_call_id links the result back to the request
    messages.append(ToolMessage(content=str(output), tool_call_id=call["id"]))

# --- STEP 3: send results back; the model writes the final answer ---
final = llm_with_tools.invoke(messages)
print(final.content)
# "It's currently 34°C and sunny in Lahore. And 17 × 24 = 408."
```

Notice: **the model got the arithmetic right**, because it didn't do the arithmetic — the calculator did. This is why tools matter.

KEY IDEA

That hand-written loop is *exactly* what an "agent" is. An agent is a loop: call the model → if it wants tools, run them → feed results back → repeat until it gives a final answer. **LangGraph (Part IV) is a framework for writing that loop properly.** You have now already understood agents.

3.7 Component 5 — Memory

Since the model has no memory, "memory" in LangChain means: *a strategy for deciding what to put back into the message list*. There are three strategies, and the tradeoff between them is a favourite interview topic.

Strategy	How	Trade-off
Buffer	Keep the entire conversation. Send everything, every time.	Perfect recall. But grows without limit — eventually blows the context window and gets expensive.
Window (buffer window)	Keep only the last *k* turns.	Bounded cost. But the model forgets anything older than *k* turns — including the user's name if they said it early.
Summary	Periodically ask the model to summarise old turns; keep the summary plus recent turns verbatim.	Bounded cost *and* long-range recall. But costs an extra LLM call, and details get lost in summarisation.

The modern LangChain approach uses `RunnableWithMessageHistory`, keyed by a `session_id` so different users get different histories:

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="history"),   # ← past turns injected here
    ("human", "{input}"),
])

chain = prompt | llm | StrOutputParser()

# In production this would be Redis / Postgres, not a Python dict
store = {}
def get_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

chat = RunnableWithMessageHistory(
    chain,
    get_history,
    input_messages_key="input",
    history_messages_key="history",
)

cfg = {"configurable": {"session_id": "user_42"}}

print(chat.invoke({"input": "My name is Ali and I love cricket."}, config=cfg))
# "Nice to meet you, Ali! Cricket is a great sport..."

print(chat.invoke({"input": "What's my name and what do I like?"}, config=cfg))
# "Your name is Ali, and you love cricket."   ← it remembered!

# A different session_id has NO memory of Ali
cfg2 = {"configurable": {"session_id": "user_99"}}
print(chat.invoke({"input": "What's my name?"}, config=cfg2))
# "I don't have that information — you haven't told me your name."
```

INTERVIEW QUESTION

Q: How would you handle a chatbot conversation that grows longer than the context window?

Three options, escalating in sophistication. **(1) Windowing** — keep only the last *k* turns. Simple and cheap, but loses early context. **(2) Summarisation** — periodically compress older turns into a running summary and keep recent turns verbatim. This preserves long-range context at the cost of one extra LLM call and some detail loss. **(3) Retrieval over history** — embed every past turn into a vector store, and at each new message retrieve only the semantically relevant past turns. This scales to arbitrarily long conversations and is what serious production systems do. In practice I'd combine (2) and (3): a running summary for continuity, plus retrieval for specific recall.

3.8 Component 6 — Loaders, Splitters, Retrievers

These three exist to get your documents into a form the model can use. They are the front half of RAG.

Document loaders — get the text in

```
from langchain_community.document_loaders import (
    PyPDFLoader, TextLoader, WebBaseLoader, CSVLoader, DirectoryLoader
)

docs = PyPDFLoader("handbook.pdf").load()
print(len(docs))              # one Document per page
print(docs[0].page_content[:200])
print(docs[0].metadata)       # {'source': 'handbook.pdf', 'page': 0}
```

Every loader returns a list of `Document` objects. A `Document` has exactly two fields:

- `page_content` — the text, as a string.
- `metadata` — a dict. Source file, page number, author, date, anything.

KEY IDEA

Metadata is not an afterthought. It is how you build citations ("this answer came from page 14 of the handbook"), and how you filter searches ("only search documents from 2024"). Populate it richly at load time — you cannot easily add it later.

Text splitters — cut it into chunks

You cannot embed a whole book as one vector — the meaning would be averaged into mush, and it wouldn't fit in a prompt anyway. So you split documents into **chunks**.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,        # target characters per chunk
    chunk_overlap=200,      # characters repeated between adjacent chunks
    separators=["\n\n", "\n", ". ", " ", ""],   # try these IN ORDER
)

chunks = splitter.split_documents(docs)
print(f"{len(docs)} pages → {len(chunks)} chunks")
```

Why "Recursive"? It tries to split on paragraph breaks first. If a piece is still too big, it tries single newlines. Then sentences. Then spaces. Then, as a last resort, mid-word. This means it *respects natural boundaries* wherever possible — a paragraph stays whole rather than being guillotined at character 1000.

Why overlap? Because a sentence that straddles a chunk boundary would otherwise be cut in half, and neither half would make sense. Overlap gives each chunk a bit of its neighbour's context.

> ANALOGY
>
> Imagine tearing a newspaper article into strips to hand out. If you tear at random, someone gets "...the suspect was later found" and someone else gets "innocent by the jury." Neither is useful. Overlapping strips means each person gets enough surrounding text for their piece to make sense on its own.

Retrievers — get the right chunks back

A retriever is any object with one job: given a query string, return relevant documents. Every vector store can produce one.

```
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4},   # return the top 4 most similar chunks
)

results = retriever.invoke("What is the parental leave policy?")
for doc in results:
    print(f"[page {doc.metadata['page']}] {doc.page_content[:100]}...")
```

A retriever is a `Runnable`, which is why it drops directly into an LCEL chain with a `|`. Everything connects.

4. Part III — RAG (Retrieval-Augmented Generation)

4.1 Why RAG exists

Your model doesn't know your company's HR handbook. You want it to answer questions about it. The handbook is 400 pages. You cannot paste 400 pages into a prompt — it exceeds the context window, it would cost a fortune per query, and even if it fit, model accuracy degrades when it has to find one fact buried in a mountain of irrelevant text (the "lost in the middle" problem).

RAG's solution: don't send the whole handbook. Send only the three paragraphs that actually answer the question.

The name spells out the process:

- **R**etrieve — find the relevant chunks
- **A**ugment — insert them into the prompt
- **G**enerate — let the model answer using them

> KEY IDEA
>
> RAG turns a **generation** problem into a **search** problem plus a **summarisation** problem. And that means **the quality of your RAG system is almost entirely determined by the quality of your retrieval, not your model.** A brilliant model given the wrong three paragraphs will give a confident wrong answer. A mediocre model given the right paragraph will give a correct one. When your RAG system is bad, fix retrieval first. Interviewers love this point.

4.2 The two pipelines

Every RAG system has two distinct phases that run at different times. Confusing them is the number one beginner error.

Pipeline A — Indexing (runs once, offline, ahead of time)

```
Raw documents (PDF, HTML, CSV, database rows)
        │
        ▼  LOAD          → list of Document objects
        │
        ▼  SPLIT         → many small chunks with overlap
        │
        ▼  EMBED         → each chunk becomes a vector of floats
        │
        ▼  STORE         → vectors + text + metadata into a vector DB
        │
        ▼
   Vector database, ready to search
```

Pipeline B — Querying (runs on every user question, live)

```
User question: "What is the parental leave policy?"
        │
        ▼  EMBED         → the question becomes a vector
        │                  (SAME embedding model as indexing!)
        │
        ▼  SEARCH        → find the k nearest chunk-vectors
        │
        ▼  AUGMENT       → paste those chunks into a prompt template
        │
        ▼  GENERATE      → LLM answers using ONLY those chunks
        │
        ▼
   Answer + citations
```

COMMON MISTAKE

Re-indexing your entire document set on every user query. Indexing is **slow and expensive** — it embeds every chunk. Do it once, persist the vector store to disk, and load it at startup. If your RAG demo takes 30 seconds per question, this is why.

4.3 A complete, working RAG system

Here is the whole thing, end to end, in about 40 lines. Read it twice.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# ========== PIPELINE A: INDEXING (run once) ==========

# 1. LOAD
docs = PyPDFLoader("employee_handbook.pdf").load()

# 2. SPLIT
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)
print(f"Split {len(docs)} pages into {len(chunks)} chunks")

# 3+4. EMBED and STORE (Chroma does both in one call)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",   # saves to disk — reuse it next time!
)

# ========== PIPELINE B: QUERYING (run per question) ==========

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_template("""
Answer the question using ONLY the context below.
If the context does not contain the answer, say "I don't know" —
do not use outside knowledge and do not guess.

Context:
{context}

Question: {question}

Answer:""")

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

def format_docs(docs):
    return "\n\n---\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

print(rag_chain.invoke("How many days of parental leave do I get?"))
```

Three things in that prompt template deserve your attention, because they are what separates a toy RAG system from a real one:

- **"using ONLY the context below"** — grounds the model. Without this it will happily blend in its own training knowledge, and you cannot tell which parts came from your documents.

- **"say I don't know"** — gives the model permission to fail. Without an explicit escape hatch, models will invent an answer rather than admit ignorance.
- `temperature=0` — the same question must give the same answer. Non-negotiable for factual systems.

Adding citations

Citations are what make a RAG system trustworthy enough to actually deploy. You get them from metadata:

```
def answer_with_sources(question: str):
    docs = retriever.invoke(question)
    context = format_docs(docs)
    answer = (prompt | llm | StrOutputParser()).invoke(
        {"context": context, "question": question}
    )
    sources = sorted({
        f"{d.metadata['source']} p.{d.metadata['page'] + 1}"
        for d in docs
    })
    return {"answer": answer, "sources": sources}

result = answer_with_sources("What is the remote work policy?")
print(result["answer"])
print("\nSources:", ", ".join(result["sources"]))
# Sources: employee_handbook.pdf p.14, employee_handbook.pdf p.15
```

4.4 Chunking — the decision that matters most

Chunk size is the highest-leverage knob in your entire RAG system, and it is a classic interview question.

Chunk size	Pro	Con
Small (200–500 chars)	Very precise retrieval. The matched chunk is almost entirely relevant — little noise.	Context is fragmented. An answer spanning two chunks may retrieve only one, giving a half-answer.
Large (1500–3000 chars)	Each chunk is self-contained. More likely to hold a complete answer.	Retrieval gets fuzzy — the chunk's embedding averages many topics, so it matches everything vaguely and nothing precisely. Also wastes context tokens on irrelevant text.

KEY IDEA

The right chunk size depends on your **content structure**. FAQ documents → chunk per question (small). Legal contracts → chunk per clause. Narrative prose → 1000 chars with 200 overlap is a good default. **There is no universal right answer — this is an empirical question you must test.** Saying that in an interview is a strong signal of maturity.

Beyond fixed-size: semantic and structural chunking

```
# Structure-aware: split Markdown by its headers, keeping them as metadata
from langchain.text_splitter import MarkdownHeaderTextSplitter

splitter = MarkdownHeaderTextSplitter(headers_to_split_on=[
    ("#",   "chapter"),
    ("##",  "section"),
    ("###", "subsection"),
])
# Now each chunk carries {'chapter': 'Benefits', 'section': 'Leave'} in metadata
# → you can filter searches by section, AND the model sees the heading context

# Semantic chunking: split where the MEANING shifts, not at a fixed length
from langchain_experimental.text_splitter import SemanticChunker

splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",
)
# Embeds each sentence; cuts where consecutive sentences become dissimilar.
# Slower and costlier, but produces topically coherent chunks.
```

4.5 What breaks in RAG (and how to fix it)

This table is worth memorising. Being able to *debug* RAG, not just build it, is what distinguishes a strong candidate.

Symptom	Root cause	Fix
"I don't know" for questions clearly in the docs	**Retrieval failure** — the right chunk never made it into the top-k	Increase `k`. Add hybrid search. Try query rewriting. **First, always: print the retrieved chunks and look at them.**
Answer is correct but incomplete	The answer spans multiple chunks; only one was retrieved	Increase chunk size or overlap. Use a parent-document retriever (search small, return large).
Answer contains facts not in the docs	**Hallucination** — model fell back on training knowledge	Strengthen the "use ONLY the context" instruction. Set `temperature=0`. Add a verification step.
Retrieves the right topic, wrong specifics	Semantic search is bad at exact terms — product codes, names, IDs, acronyms	**Hybrid search** — combine vector search with BM25 keyword search.
Answers wrong on follow-up questions	"What about *his* salary?" is embedded literally — "his" has no meaning to the retriever	**Query rewriting**: use the LLM to rewrite the follow-up into a standalone question first.
Slow and expensive	Too many chunks, too large, or `k` too high	Reduce `k`. Add a reranker so you can retrieve 20 and pass only the best 4 to the LLM.

Fix 1 — Hybrid search

Vector search finds meaning. Keyword search finds exact strings. Neither is sufficient alone — you want both.

```
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever

bm25 = BM25Retriever.from_documents(chunks)   # keyword-based
bm25.k = 4
vector = vectorstore.as_retriever(search_kwargs={"k": 4})  # meaning-based

hybrid = EnsembleRetriever(
    retrievers=[bm25, vector],
    weights=[0.4, 0.6],     # tune this on your own data
)
# Now "error code XR-4471" is found by BM25, and
# "how do I fix the login problem" is found by vector search.
```

Fix 2 — Reranking

Retrieve broadly, then use a smarter (slower) model to reorder and keep only the best. This is the single highest-ROI

upgrade to a basic RAG system.

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain_cohere import CohereRerank

reranker = CohereRerank(model="rerank-english-v3.0", top_n=4)

compressed = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=vectorstore.as_retriever(search_kwargs={"k": 25}),
)
# Fetch 25 candidates cheaply → rerank → pass only the best 4 to the LLM.
# Better accuracy AND fewer tokens sent to the expensive model.
```

Fix 3 — Query rewriting for conversations

```
rewrite_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Given the chat history and a follow-up question, rewrite the follow-up "
     "as a standalone question that makes sense on its own. Do not answer it."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])
rewriter = rewrite_prompt | llm | StrOutputParser()

# History: "Tell me about Ali Khan" / "He is a senior engineer..."
# Follow-up: "What is his salary?"
# Rewritten: "What is Ali Khan's salary?"   ← NOW it can be retrieved
```

INTERVIEW QUESTION

Q: Your RAG system returns irrelevant answers. Walk me through debugging it.

First, isolate the failure. RAG has two halves and I need to know which one broke. I'd print the retrieved chunks for the failing query. **If the right chunk isn't there, it's a retrieval problem.** If the right chunk *is* there but the answer is still wrong, it's a generation problem.
For a **retrieval** problem: increase `k`; check that indexing and querying use the same embedding model; add hybrid search if the query has exact terms like IDs or codes; check whether chunking split the answer across a boundary; add a reranker.
For a **generation** problem: strengthen the "answer only from context" instruction; set temperature to 0; check whether the context is so long the answer is getting lost in the middle; consider passing fewer, better chunks.

INTERVIEW QUESTION

Q: How do you evaluate a RAG system?

You evaluate the two halves separately. For **retrieval**, build a small labelled set of questions with their known correct source chunks, then measure **hit rate** (was the correct chunk in the top-k?) and **MRR** (how high up was it?). For **generation**, use the RAGAS metrics: **faithfulness** (is every claim in the answer supported by the retrieved context — i.e. did it hallucinate?), **answer relevance** (does it actually address the question?), and **context precision** (how much of what we retrieved was actually useful?). The key insight is that a single end-to-end accuracy number tells you nothing actionable — you must know *which half* is failing.

5. Part IV — LangGraph

5.1 Where LangChain runs out of road

An LCEL chain is a **straight line**. `prompt | llm | parser` goes one direction, and it always takes the same path. That is perfect for a fixed pipeline.

But now consider a research agent. It should:

1. Read the question.
2. Decide it needs to search the web.
3. Search.
4. Read the results and decide they're insufficient.
5. Search again with a better query.
6. Now decide it has enough. Answer.

How many times does that loop run? **You don't know.** It depends on what the model decides at runtime. You cannot express "loop an unknown number of times, deciding each iteration whether to continue" with a pipe operator. A pipe is a DAG — a directed *acyclic* graph. It has no cycles by definition.

> KEY IDEA — THE ONE-LINE ANSWER TO "WHY LANGGRAPH?"
>
> **LangChain chains are DAGs — they cannot loop. LangGraph adds cycles and shared state, which is exactly what agents need.** Chains are for fixed pipelines you design. Graphs are for dynamic control flow the model decides.

5.2 The three concepts

LangGraph reframes your application as a **state machine**. There are only three things to learn, and honestly the third one is the only tricky bit.

Concept 1 — State

The **State** is a single dictionary that flows through the entire graph. It holds everything: the message history, retrieved documents, a retry counter, whatever you need.

```
from typing import TypedDict, Annotated, List
from langchain_core.messages import BaseMessage
import operator

class AgentState(TypedDict):
    # Annotated[type, reducer] — the reducer says HOW to merge updates
    messages: Annotated[List[BaseMessage], operator.add]   # APPEND to the list
    documents: List[str]                                    # OVERWRITE (default)
    iterations: int                                         # OVERWRITE (default)
```

The reducer is the one genuinely subtle idea in LangGraph, so slow down here.

When a node returns `{"messages": [new_msg]}`, what should happen to the existing messages?

- **Without a reducer** (default): the new value *replaces* the old. Your entire message history is destroyed and replaced by one message.
- **With** `operator.add`: the new value is *appended*. History is preserved.

COMMON MISTAKE — THIS ONE BITES EVERYONE

Forgetting `Annotated[List[BaseMessage], operator.add]` on the messages field. Your agent will appear to have amnesia, forgetting everything after each node runs, and you will spend an hour confused. **Message lists almost always want `operator.add`**. LangGraph ships a prebuilt `add_messages` reducer that is smarter — it also de-duplicates by message ID, so use that in real code.

Concept 2 — Nodes

A node is **a plain Python function.** That's all. It takes the state, does something, and returns a *partial* update to the state.

```
def call_model(state: AgentState) -> dict:
    """A node. Takes state, returns a PARTIAL update."""
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}     # ← appended, thanks to the reducer

def call_tools(state: AgentState) -> dict:
    """Another node. Runs whatever tools the model asked for."""
    last_msg = state["messages"][-1]
    outputs = []
    for call in last_msg.tool_calls:
        result = tool_registry[call["name"]].invoke(call["args"])
        outputs.append(ToolMessage(content=str(result), tool_call_id=call["id"]))
    return {"messages": outputs}
```

Note that a node returns *only the keys it changed*. It doesn't need to return the whole state — LangGraph merges the partial update in for you, using the reducers.

Concept 3 — Edges

Edges determine what runs next. There are two kinds, and the difference is everything.

```
# NORMAL EDGE — always go from A to B
graph.add_edge("tools", "agent")

# CONDITIONAL EDGE — a FUNCTION decides where to go next
def should_continue(state: AgentState) -> str:
    """This is the agent's 'if' statement. It returns a NODE NAME."""
    last_msg = state["messages"][-1]
    if last_msg.tool_calls:      # the model asked for a tool
        return "tools"           # → go run them
    return END                   # → no tools requested, we're done

graph.add_conditional_edges("agent", should_continue, {
    "tools": "tools",
    END: END,
})
```

KEY IDEA

The conditional edge *is* the loop. Look at the shape: `agent` → conditional edge → `tools` → normal edge back to `agent`. That cycle runs as many times as the model needs, and terminates the moment the model stops requesting tools. This is the entire agent pattern.

5.3 A complete agent, from scratch

Here it is — a full tool-using agent in LangGraph. This is the code to be able to write from memory in an interview.

```python
from typing import TypedDict, Annotated, List
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage, HumanMessage, ToolMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# ── 1. TOOLS ──────────────────────────────────────────
@tool
def search(query: str) -> str:
    """Search the web for current information."""
    return f"[search results for '{query}']"

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression. Use for ALL arithmetic."""
    return str(eval(expression, {"__builtins__": {}}, {}))

tools = [search, calculator]
tool_registry = {t.name: t for t in tools}
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)

# ── 2. STATE ──────────────────────────────────────────
class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

# ── 3. NODES ──────────────────────────────────────────
def agent_node(state: AgentState):
    return {"messages": [llm.invoke(state["messages"])]}

def tool_node(state: AgentState):
    results = []
    for call in state["messages"][-1].tool_calls:
        output = tool_registry[call["name"]].invoke(call["args"])
        results.append(ToolMessage(content=str(output), tool_call_id=call["id"]))
    return {"messages": results}

# ── 4. THE ROUTER (conditional edge) ──────────────────
def should_continue(state: AgentState):
    return "tools" if state["messages"][-1].tool_calls else END

# ── 5. WIRE THE GRAPH ─────────────────────────────────
builder = StateGraph(AgentState)

builder.add_node("agent", agent_node)
builder.add_node("tools", tool_node)

builder.add_edge(START, "agent")                    # entry point
builder.add_conditional_edges("agent", should_continue)  # the branch
builder.add_edge("tools", "agent")                  # ← THE LOOP

graph = builder.compile()

# ── 6. RUN IT ─────────────────────────────────────────
result = graph.invoke({
    "messages": [HumanMessage(content="What is 4,271 * 89? Then search for LangGraph.")]
})

for msg in result["messages"]:
    print(f"{msg.type.upper():10} {msg.content[:80]}")
```

Trace the execution in your head. `START` → `agent` (model requests calculator) → conditional says `"tools"` → `tools` runs it → edge goes back to `agent` → model now requests search → conditional says `"tools"` again → `tools` → back to `agent` → model has everything, writes final answer → conditional says `END`. Three trips through the agent node, decided entirely at runtime.

SHORTCUT YOU SHOULD KNOW EXISTS

LangGraph ships a prebuilt version of exactly this: `from langgraph.prebuilt import create_react_agent`, then `agent = create_react_agent(llm, tools)`. One line. **But you must be able to write it by hand for an interview** — "I use the prebuilt one" is a much weaker answer than being able to explain the loop.

5.4 Checkpointers — memory, pause/resume, and time travel

Because the state is a single explicit object, LangGraph can **snapshot it after every single node**. Add a checkpointer and you get four capabilities for free.

```
from langgraph.checkpoint.memory import MemorySaver
# For production: from langgraph.checkpoint.postgres import PostgresSaver

memory = MemorySaver()
graph = builder.compile(checkpointer=memory)

# thread_id identifies a conversation — like a session ID
config = {"configurable": {"thread_id": "conversation_1"}}

graph.invoke({"messages": [HumanMessage("My name is Ali.")]}, config)
graph.invoke({"messages": [HumanMessage("What's my name?")]}, config)
# → "Your name is Ali."  ← persisted automatically, no memory code written

# A different thread has no shared memory
config2 = {"configurable": {"thread_id": "conversation_2"}}
graph.invoke({"messages": [HumanMessage("What's my name?")]}, config2)
# → "I don't know your name."
```

Capability	What it gives you
Persistence	Conversations survive process restarts. Point the checkpointer at Postgres and your agent remembers across deployments.
Pause / resume	Stop mid-execution, come back hours later, continue from exactly where you left off.
Human-in-the-loop	Interrupt before a dangerous node, let a human approve, then continue. Essential for agents that spend money or send emails.
Time travel	Rewind to any previous checkpoint and re-run from there with different inputs. Invaluable for debugging.

Human-in-the-loop — the killer feature

```
# Stop the graph BEFORE the tools node executes
graph = builder.compile(
    checkpointer=memory,
    interrupt_before=["tools"],
)

config = {"configurable": {"thread_id": "session_1"}}

# Run — it will PAUSE before running any tool
graph.invoke({"messages": [HumanMessage("Delete all records older than 2020.")]}, config)

# Inspect what it's about to do
snapshot = graph.get_state(config)
pending = snapshot.next                                  # ('tools',)
proposed = snapshot.values["messages"][-1].tool_calls
print(f"Agent wants to run: {proposed}")

approval = input("Approve? (y/n): ")
if approval == "y":
    graph.invoke(None, config)     # passing None = "resume from checkpoint"
else:
    print("Rejected. Nothing was executed.")
```

INTERVIEW QUESTION

Q: Why would you use LangGraph instead of a LangChain agent?

Four reasons. **(1) Cycles** — LCEL chains are DAGs and cannot loop; agents fundamentally need loops. **(2) Explicit state** — in a LangChain `AgentExecutor`, the loop is hidden inside a black box; in LangGraph the state is a dict I define, so I can inspect it, add fields like retry counters, and actually debug it. **(3) Control** — conditional edges let me hard-code guardrails, like "after 5 iterations, force a stop" — I'm not just hoping the model behaves. **(4) Persistence** — the checkpointer gives me durable memory, pause/resume, human-in-the-loop approval, and time-travel debugging, none of which I get from a chain.

5.5 Multi-agent patterns

Once you have graphs, one node can be an *entire other agent*. Three architectures are worth knowing by name:

Pattern	Structure	Use when
Supervisor	One "manager" LLM routes each task to a specialist agent (researcher, coder, writer). Specialists report back to the supervisor.	Most common. Clear division of labour, and the supervisor's routing decision is easy to inspect.
Hierarchical	Supervisors of supervisors. A top-level manager routes to team leads, who route to workers.	Very complex domains where one supervisor would have too many options to choose between.
Network / swarm	Any agent can hand off to any other agent directly. No central coordinator.	Rarely. Flexible but hard to reason about, hard to debug, and prone to infinite handoff loops.

```
from typing import Literal

def supervisor(state) -> Command[Literal["researcher", "coder", END]]:
    """Routes to the right specialist based on the task."""
    decision = router_llm.invoke(state["messages"])   # structured output: which agent?
    return Command(goto=decision.next_agent)

builder.add_node("supervisor", supervisor)
builder.add_node("researcher", research_agent)   # ← itself a compiled graph!
builder.add_node("coder", coding_agent)

builder.add_edge("researcher", "supervisor")     # specialists report back
builder.add_edge("coder", "supervisor")
```

COMMON MISTAKE

Always set a **recursion limit**. Without one, an agent stuck in a "search → still not enough → search again" loop will run until your API bill is enormous.

`graph.invoke(state, {"recursion_limit": 15})`

Better still, put an iteration counter in your state and have the conditional edge force `END` when it exceeds a threshold. **Mentioning this unprompted in an interview is a strong signal.**

5.6 The decision table — chain or graph?

Use a LangChain chain when...	Use a LangGraph graph when...
The steps are fixed and known in advance	The path depends on what the model decides at runtime
Data flows one direction, no loops	You need to loop — retry, refine, iterate until good enough
Each step's input is just the previous step's output	Multiple nodes need to read and write shared state
Examples: summarisation, translation, classification, **basic RAG**	Examples: **tool-using agents**, self-correcting RAG, multi-agent systems, anything needing human approval

KEY IDEA

They are not rivals — **they compose**. Your LangGraph *nodes* are usually calling LangChain *chains* inside them. LangChain gives you the components; LangGraph gives you the control flow between them.

6. Part V — Vector Databases

6.1 What a vector database is

You have 100,000 chunks, each embedded as a 1,536-dimensional vector. A user asks a question, you embed it, and now you must find the 4 closest vectors out of 100,000.

The naive approach: compute cosine similarity against all 100,000 and sort. That's a **brute-force k-NN search**, and it works fine — until you have 10 million vectors, at which point every query takes seconds. Too slow.

A vector database is a database whose primary index is built for fast nearest-neighbour search in high-dimensional space. That's the whole definition.

> ANALOGY
>
> A normal database index is like a book's alphabetical index — it answers "where is the word 'mitochondria'?" instantly. But it cannot answer "what pages are *about the same topic* as page 47?" A vector index answers *that* question. It's less like an index and more like a map: things that are near each other on the map are similar.

ANN — the trick that makes it fast

Vector DBs don't find the *exact* nearest neighbours. They use **Approximate Nearest Neighbour (ANN)** algorithms, which find *almost certainly* the right neighbours, thousands of times faster.

The dominant algorithm is **HNSW** (Hierarchical Navigable Small World). Intuitively: it builds a multi-layer graph. The top layer has few nodes with long-range links — you use it to jump roughly into the right region fast. Lower layers have more nodes with shorter links — you use them to refine. It's like navigating with a country map, then a city map, then a street map.

> KEY IDEA
>
> ANN trades a small amount of **recall** (you might miss the true 4th-nearest neighbour and get the 5th instead) for an enormous gain in **speed**. For RAG this is a fantastic trade — you don't need the mathematically optimal chunk, you need a good one, fast. Every vector DB exposes a knob to tune this trade-off.

Similarity metrics

Metric	Measures	Use when
Cosine similarity	The angle between vectors. Ignores magnitude.	**The default for text.** You want "same meaning" regardless of document length.
Dot product	Angle *and* magnitude.	When vectors are already normalised (then it's identical to cosine, but faster).
Euclidean (L2)	Straight-line distance.	Image embeddings, and cases where magnitude carries meaning.

6.2 ChromaDB — the one you learn on

Chroma is open-source, runs in-process (no server to start), and persists to a local folder. It is the correct choice for learning, prototyping, and any project under roughly a million vectors.

```
import chromadb

# Persistent client — saves to disk
client = chromadb.PersistentClient(path="./chroma_data")

collection = client.get_or_create_collection(
    name="handbook",
    metadata={"hnsw:space": "cosine"},   # the similarity metric
)

# Add documents. Chroma embeds them FOR you if you don't supply vectors.
collection.add(
    documents=[
        "Employees get 20 days of annual leave per year.",
        "Parental leave is 12 weeks, fully paid.",
        "Remote work is allowed 3 days per week.",
    ],
    metadatas=[
        {"section": "leave",  "page": 12},
        {"section": "leave",  "page": 14},
        {"section": "remote", "page": 22},
    ],
    ids=["c1", "c2", "c3"],       # you must supply unique IDs
)

# Query
results = collection.query(
    query_texts=["How much time off do new parents get?"],
    n_results=2,
)
print(results["documents"][0][0])
# "Parental leave is 12 weeks, fully paid."   ← found WITHOUT sharing any keywords!

# Metadata filtering — search only within a section
results = collection.query(
    query_texts=["time off"],
    n_results=2,
    where={"section": "leave"},           # ← pre-filter before the vector search
)
```

Notice the query used the words "time off" and "new parents", while the document said "parental leave" and "12 weeks". No keyword overlap at all — and it still matched. **That is semantic search, and it is the entire point of vector databases.**

Via LangChain, the same thing is even shorter:

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(),
    persist_directory="./chroma_db",
)

# Later — reload from disk instead of re-embedding everything
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=OpenAIEmbeddings(),
)

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

6.3 Pinecone — the managed production option

Pinecone is a fully-managed cloud vector database. You don't run a server; you call an API. It scales to billions of vectors and is the standard choice when you need production reliability without an infrastructure team.

```
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key="YOUR_KEY")

pc.create_index(
    name="handbook",
    dimension=1536,          # MUST match your embedding model exactly
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1"),
)
index = pc.Index("handbook")

# Upsert — insert or update. You provide the vectors yourself.
index.upsert(vectors=[
    {"id": "c1",
     "values": embed("Employees get 20 days of annual leave."),
     "metadata": {"section": "leave", "page": 12, "text": "Employees get 20 days..."}},
    {"id": "c2",
     "values": embed("Parental leave is 12 weeks, fully paid."),
     "metadata": {"section": "leave", "page": 14, "text": "Parental leave is..."}},
], namespace="v1")

results = index.query(
    vector=embed("time off for new parents"),
    top_k=3,
    include_metadata=True,
    filter={"section": {"$eq": "leave"}},
    namespace="v1",
)
for match in results["matches"]:
    print(f"{match['score']:.3f}  {match['metadata']['text']}")
```

COMMON MISTAKE

Pinecone stores **only the vector**, not your text. If you forget to put the original text in `metadata`, you'll get back IDs and similarity scores with no way to recover the actual content. **Always store the source text in metadata.**

TWO PINECONE CONCEPTS WORTH NAMING IN AN INTERVIEW

Namespaces — logical partitions inside one index. Perfect for multi-tenancy: give each customer a namespace and their data is fully isolated, with no risk of one tenant's search returning another's documents. **Upsert** — insert-or-update by ID. This means re-indexing a changed document just overwrites its vectors; you don't need to delete first.

6.4 Weaviate — the hybrid-search option

Weaviate is open-source, can be self-hosted or cloud-managed, and its distinguishing feature is **first-class hybrid search** — it combines vector search and BM25 keyword search natively, with one tuning parameter.

```
import weaviate
from weaviate.classes.query import HybridFusion

client = weaviate.connect_to_local()

collection = client.collections.get("Handbook")

# Pure vector search
collection.query.near_text(query="time off for parents", limit=3)

# HYBRID search — Weaviate's signature feature
results = collection.query.hybrid(
    query="parental leave policy XR-4471",
    alpha=0.5,          # 0.0 = pure keyword (BM25), 1.0 = pure vector
    limit=3,
    fusion_type=HybridFusion.RELATIVE_SCORE,
)
for obj in results.objects:
    print(obj.properties["text"])
```

WHY HYBRID SEARCH MATTERS — AND WHY INTERVIEWERS ASK ABOUT IT

Vector search is *bad at exact strings*. Product codes, error codes, person names, and acronyms have weak semantic signal — "XR-4471" doesn't *mean* anything, so its embedding is near-random. Keyword search nails these but misses paraphrases. **Hybrid gets both.** The `alpha` parameter is the dial between them, and tuning it on your own data is a real, practical skill.

6.5 Choosing between them

	ChromaDB	Pinecone	Weaviate
Hosting	Embedded / local	Fully managed cloud	Self-host or cloud
Setup time	Zero — `pip install`	Minutes — sign up, get key	Docker, or cloud
Cost	Free	Paid, usage-based	Free if self-hosted
Scale	Up to ~1M vectors	Billions	Hundreds of millions
Hybrid search	No (build it yourself)	Yes (sparse-dense)	**Yes — best in class**
Best for	Learning, prototypes, small apps	Production, no ops team	Production, want control, need hybrid

INTERVIEW QUESTION

Q: How would you pick a vector database for a project?

I'd ask three questions. **Scale?** Under a million vectors, Chroma or even FAISS in memory is plenty — you don't need a distributed system. Above that, Pinecone or Weaviate. **Ops capacity?** If there's no infra team, Pinecone's managed service removes an entire category of problems. If the team can run Kubernetes and wants to avoid vendor lock-in or per-query costs, self-hosted Weaviate or Qdrant. **Do we need hybrid search?** If the domain has exact identifiers — SKUs, error codes, part numbers — then yes, and Weaviate's native hybrid gives it to me out of the box.

The honest answer I'd add: **start with Chroma, because it takes zero setup, and swap later if you outgrow it.** LangChain's `VectorStore` interface means changing DB is a few lines. Choosing infrastructure before you have a working prototype is premature optimisation.

7. Part VI — LlamaIndex

7.1 LlamaIndex vs LangChain

LlamaIndex is a competing framework. The distinction is genuinely simple:

	LangChain	LlamaIndex
Philosophy	A general toolkit for *any* LLM app — agents, chains, tools, RAG.	A specialised toolkit for *connecting LLMs to data*. RAG is the whole product.
Strength	Flexibility, agents, orchestration, huge integration ecosystem.	Ingestion and retrieval. More advanced indexing strategies, better defaults for RAG.
Feel	More code, more control, more assembly required.	Fewer lines to a good RAG system. More opinionated.

> KEY IDEA — THE INTERVIEW ANSWER
>
> **"LangChain is a general-purpose orchestration framework; LlamaIndex is specialised for data ingestion and retrieval."** If the app is fundamentally "search my documents", LlamaIndex gets there with less code and better out-of-the-box retrieval. If the app is "an agent that uses tools and makes decisions", LangChain/LangGraph. **They are also interoperable** — a common production pattern is LlamaIndex for the retrieval layer, exposed as a LangChain retriever inside a LangGraph agent.

7.2 The core objects

LlamaIndex	LangChain equivalent	What it is
`Document`	`Document`	A raw source file's text plus metadata.
`Node`	chunk (a `Document`)	A chunk. But **LlamaIndex Nodes know their neighbours** — they carry PREV/NEXT/PARENT relationships. This enables retrieval tricks LangChain has to work harder for.
`VectorStoreIndex`	`VectorStore`	The searchable index.
`QueryEngine`	RAG chain	Ask a question, get an answer. One-shot, no memory.
`ChatEngine`	RAG chain + memory	Same, but conversational.
`Retriever`	`Retriever`	Returns nodes, doesn't generate.

7.3 RAG in five lines

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("./data").load_data()   # loads EVERY file type
index = VectorStoreIndex.from_documents(documents)        # chunks, embeds, stores
query_engine = index.as_query_engine()

response = query_engine.query("What is the parental leave policy?")
print(response)
print(response.source_nodes)     # citations, for free
```

That is a complete RAG system. Compare it to the 40-line LangChain version in Part III — **this is LlamaIndex's entire pitch.** The trade-off is that the defaults are chosen for you; when you need to change one, you must learn LlamaIndex's abstraction for it.

Persisting and configuring

```
from llama_index.core import Settings, StorageContext, load_index_from_storage
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.core.node_parser import SentenceSplitter

# Global config — set once
Settings.llm = OpenAI(model="gpt-4o-mini", temperature=0)
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small")
Settings.node_parser = SentenceSplitter(chunk_size=512, chunk_overlap=50)

index = VectorStoreIndex.from_documents(documents)
index.storage_context.persist(persist_dir="./storage")     # save

# Later — reload without re-embedding
ctx = StorageContext.from_defaults(persist_dir="./storage")
index = load_index_from_storage(ctx)
```

7.4 The features LlamaIndex is actually good at

Chat engine — RAG with memory, in one line

```
chat_engine = index.as_chat_engine(
    chat_mode="condense_plus_context",   # rewrites follow-ups into standalone questions
    verbose=True,
)

print(chat_engine.chat("Tell me about the leave policy."))
print(chat_engine.chat("How does that compare to sick leave?"))
# ↑ "that" is automatically resolved — the engine rewrites the query before retrieving.
# In LangChain you'd write the query-rewriting chain yourself (see Part III, Fix 3).
```

Response modes — how multiple chunks get combined

Mode	How	Cost
`compact` *(default)*	Stuff as many chunks as fit into one prompt.	1 LLM call. Cheap and fast.
`refine`	Answer using chunk 1. Then show chunk 2 and ask it to improve the answer. Repeat.	N calls. Slow, but handles more context than fits at once.
`tree_summarize`	Summarise chunks in pairs, then summarise the summaries, recursively.	N calls. Best for "summarise this whole document".

```
engine = index.as_query_engine(response_mode="tree_summarize", similarity_top_k=10)
```

Auto-merging retrieval — the genuinely clever one

This is LlamaIndex's flagship advanced feature and a great thing to mention in an interview, because it elegantly solves the chunk-size dilemma from Part III.

> THE IDEA
>
> **Index small chunks (precise retrieval), but return large chunks (complete context).** LlamaIndex builds a hierarchy: big parent nodes containing small child nodes. You search over the *children*, so retrieval is precise. But if several children of the same parent get retrieved, LlamaIndex *automatically merges them up* and hands the LLM the whole parent instead. **You get precision AND completeness, instead of trading one for the other.**

```
from llama_index.core.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.core.retrievers import AutoMergingRetriever

parser = HierarchicalNodeParser.from_defaults(chunk_sizes=[2048, 512, 128])
nodes = parser.get_nodes_from_documents(documents)

storage_context = StorageContext.from_defaults()
storage_context.docstore.add_documents(nodes)         # ALL nodes, all levels

# Index only the LEAVES (the 128-char chunks) — precise search
leaf_index = VectorStoreIndex(get_leaf_nodes(nodes), storage_context=storage_context)

retriever = AutoMergingRetriever(
    leaf_index.as_retriever(similarity_top_k=12),
    storage_context,
    verbose=True,
)
# Searches tiny chunks; returns big ones when enough siblings match.
```

Using LlamaIndex retrieval inside LangChain

```
from llama_index.core.langchain_helpers.agents import IndexToolConfig, LlamaIndexTool

tool = LlamaIndexTool.from_tool_config(IndexToolConfig(
    query_engine=index.as_query_engine(),
    name="company_handbook",
    description="Useful for questions about company policy, HR, and benefits.",
))

# This is now a normal LangChain tool — bind it to a LangGraph agent.
llm_with_tools = llm.bind_tools([tool, search, calculator])
```

This is the pattern to describe if asked "would you use both?" — **LlamaIndex as the retrieval engine, wrapped as a tool, inside a LangGraph agent.** Best of both.

8. Part VII — Python & the ML Frameworks

This part assumes you can write Python but have never trained a neural network. By the end you will understand what a tensor is, what "training" actually does, and be able to write the same model in both PyTorch and Keras.

8.1 The Python you actually need

Four language features show up constantly in ML code. If these look unfamiliar, learn them before going further.

```
# 1. List/dict comprehensions — used everywhere
squares  = [x**2 for x in range(10)]
lookup   = {t.name: t for t in tools}          # you saw this in the agent code
filtered = [d for d in docs if d.score > 0.7]

# 2. Type hints — every modern library uses them
from typing import List, Dict, Optional, TypedDict
def embed(texts: List[str]) -> List[List[float]]: ...

# 3. Decorators — @tool, @app.route, @torch.no_grad
@tool
def search(query: str) -> str: ...

# 4. Context managers — guarantee cleanup
with open("f.txt") as f:      # file closes even if an exception is raised
    data = f.read()
with torch.no_grad():         # disables gradient tracking inside the block
    predictions = model(x)
```

8.2 NumPy in five minutes

NumPy gives Python fast, multi-dimensional arrays. Every ML framework is built on this mental model, so it is the foundation.

```
import numpy as np

a = np.array([1, 2, 3])                    # shape (3,)      — 1D, a vector
b = np.array([[1, 2, 3], [4, 5, 6]])       # shape (2, 3)    — 2D, a matrix

print(b.shape)   # (2, 3)  ← ALWAYS check shapes. Most bugs are shape bugs.
print(b.dtype)   # int64

# VECTORISATION: operations apply element-wise, with NO Python loop.
# This is ~100x faster than looping, because it runs in compiled C.
print(a * 2)         # [2 4 6]
print(a + a)         # [2 4 6]
print(b @ a)         # [14 32]   ← matrix multiply. THE core operation of ML.

# Broadcasting: NumPy auto-expands mismatched shapes when it can
print(b + np.array([10, 20, 30]))    # adds the row to EVERY row of b
# [[11 22 33]
#  [14 25 36]]
```

KEY IDEA

A neural network is, mathematically, a long chain of matrix multiplications with a squashing function between them. That's why `@` is the most important operator in ML, and why GPUs — which are extremely good at multiplying big matrices in parallel — made deep learning possible.

8.3 Tensors, autograd, and the training loop

What a tensor is

A **tensor** is a NumPy array with two superpowers: it can live on a GPU, and it **remembers every operation performed on it**

so that derivatives can be computed automatically.

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
print(x.shape)          # torch.Size([3])

# Move to GPU (if available)
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)
```

What training actually is

Here is the entire idea of machine learning, stripped bare:

1. Your model has millions of numbers in it, called **parameters** (or weights). They start out random.
2. You show it an input and it produces an output. Because the weights are random, the output is garbage.
3. You compute the **loss** — a single number measuring how wrong the output was.
4. You compute the **gradient** — for *every single weight*, "if I nudge this weight up slightly, does the loss go up or down, and by how much?"
5. You nudge every weight a tiny step in the direction that *reduces* loss.
6. Repeat millions of times.

> ANALOGY — GRADIENT DESCENT
>
> You're standing on a foggy hillside and want to reach the bottom of the valley. You can't see the valley, but you can *feel the slope under your feet*. So you take a small step in the steepest downhill direction. Then you feel the slope again. Then you step again. Eventually you're at the bottom.
>
> **The hill is the loss. Your position is the set of all weights. The slope you feel is the gradient. Your step size is the learning rate.** Too big a step and you leap over the valley and end up on the opposite hillside — the loss oscillates and never converges. Too small and you'll be walking for a week.

Autograd — the automatic derivative

Step 4 above — computing the gradient for every weight — would be an appalling amount of calculus to do by hand. **Autograd does it for you.** This is the single feature that makes deep learning practical.

```
import torch

# requires_grad=True tells PyTorch: "track every operation on this tensor"
w = torch.tensor([2.0], requires_grad=True)
x = torch.tensor([3.0])

y = w * x           # y = 6
loss = y ** 2       # loss = 36

loss.backward()     # ← walks BACKWARD through the operations, applying the chain rule

print(w.grad)       # tensor([36.])
# Verify by hand: loss = (w*x)^2 = w^2 * 9
#                 d(loss)/dw = 2 * w * 9 = 2 * 2 * 9 = 36  ✓
```

> KEY IDEA
>
> As you do operations on tensors, PyTorch silently builds a graph of what you did. `.backward()` traverses that graph in reverse, applying the chain rule at each step, and deposits the derivative into `.grad` for every parameter. **This is "backpropagation."** It is nothing more than the chain rule from calculus, applied automatically, at scale.

8.4 PyTorch — the researcher's framework

PyTorch is the dominant framework in research and, increasingly, in production. Its defining feature is that **you write the training loop yourself**. This is more code, but every step is visible and changeable.

Defining a model

```
import torch
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self, in_features=784, hidden=128, out_classes=10):
        super().__init__()
        self.layer1 = nn.Linear(in_features, hidden)   # 784 → 128
        self.relu   = nn.ReLU()                        # the "squashing" function
        self.drop   = nn.Dropout(0.2)                  # regularisation
        self.layer2 = nn.Linear(hidden, out_classes)   # 128 → 10

    def forward(self, x):
        """Defines HOW data flows through the layers."""
        x = self.layer1(x)
        x = self.relu(x)
        x = self.drop(x)
        return self.layer2(x)

model = SimpleNet()
print(model)
print(f"Parameters: {sum(p.numel() for p in model.parameters()):,}")   # 101,770
```

What is `nn.Linear`? It computes `output = input @ W + b`. That's it — a matrix multiply plus a bias. `W` and `b` are the learnable parameters.

Why `ReLU`? Without it, stacking linear layers is pointless — the composition of two linear functions is just another linear function, so a 50-layer network would have exactly the same power as a 1-layer one. **ReLU** (`max(0, x)`) introduces non-linearity, which is what lets the network learn curved, complex decision boundaries. **This is a very common interview question.**

The training loop — write this out until you know it

```
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Fake data: 1000 samples, 784 features each, 10 classes
X = torch.randn(1000, 784)
y = torch.randint(0, 10, (1000,))
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)

model     = SimpleNet()
criterion = nn.CrossEntropyLoss()                       # the LOSS function
optimizer = optim.Adam(model.parameters(), lr=0.001)     # the OPTIMISER

for epoch in range(5):
    model.train()                       # enables dropout, batchnorm updates
    total_loss = 0

    for batch_X, batch_y in loader:
        # —— THE FIVE SACRED STEPS ——————————————————————
        optimizer.zero_grad()           # 1. clear old gradients
        outputs = model(batch_X)        # 2. forward pass  → predictions
        loss = criterion(outputs, batch_y)   # 3. how wrong were we?
        loss.backward()                 # 4. backward pass → compute gradients
        optimizer.step()                # 5. nudge the weights
        # ————————————————————————————————————————————
        total_loss += loss.item()

    print(f"Epoch {epoch+1}  loss: {total_loss/len(loader):.4f}")

# Evaluation
model.eval()                            # disables dropout
with torch.no_grad():                   # don't track gradients — faster, less memory
    preds = model(X).argmax(dim=1)
    print(f"Accuracy: {(preds == y).float().mean():.2%}")
```

COMMON MISTAKE — THE #1 PYTORCH BUG

Forgetting `optimizer.zero_grad()`. PyTorch **accumulates** gradients by default — it adds new gradients to whatever was already in `.grad`. If you don't clear them, batch 2's gradients get added to batch 1's, batch 3's to both, and your training silently produces nonsense. It doesn't crash. It just doesn't learn. **Interviewers ask about this precisely because it's a silent failure.**

COMMON MISTAKE #2

Forgetting `model.eval()` and `torch.no_grad()` at evaluation time. `model.eval()` turns off dropout — if you leave it on, you're randomly zeroing out neurons during testing, and your accuracy will look worse than it is. `torch.no_grad()` stops building the gradient graph, which you don't need for inference and which wastes memory.

KEY VOCABULARY

Epoch = one full pass over the training data. **Batch** = a small group of samples processed together (you don't feed one sample at a time — GPUs are parallel, so batches are far more efficient). **Learning rate** = the step size. **Optimiser** = the algorithm that decides *how* to apply the gradients; Adam is the sensible default. **Loss function** = `CrossEntropyLoss` for classification, `MSELoss` for regression.

8.5 TensorFlow & Keras — the production framework

Keras is the high-level API that runs on top of TensorFlow. Where PyTorch makes you write the training loop, Keras gives you `.fit()` — one line that does everything. Less control, far less code.

The Sequential API — for simple stacks

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),      # = nn.Linear + nn.ReLU in one
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax"),    # softmax → probabilities
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

model.summary()          # prints the architecture and parameter count

# ── The ENTIRE training loop, in one line ──
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
)

loss, acc = model.evaluate(X_test, y_test)
predictions = model.predict(X_test)
```

Compare that `.fit()` call to the 15-line PyTorch loop above. They do the same thing. **That is the entire PyTorch-vs-Keras trade-off, in one image.**

The Functional API — for anything non-linear

`Sequential` only handles a straight stack of layers. The moment you need two inputs, two outputs, or a skip connection, you need the Functional API.

```
inputs = keras.Input(shape=(784,))
x = layers.Dense(128, activation="relu")(inputs)
x = layers.Dropout(0.2)(x)

# TWO output heads from one shared body — impossible with Sequential
class_out = layers.Dense(10, activation="softmax", name="class")(x)
score_out = layers.Dense(1,  activation="sigmoid", name="score")(x)

model = keras.Model(inputs=inputs, outputs=[class_out, score_out])

model.compile(
    optimizer="adam",
    loss={"class": "sparse_categorical_crossentropy", "score": "binary_crossentropy"},
    loss_weights={"class": 1.0, "score": 0.3},
)
```

Callbacks — the thing Keras is genuinely great at

```
callbacks = [
    # Stop when validation loss stops improving; restore the best weights
    keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=3, restore_best_weights=True
    ),
    # Save the best model to disk
    keras.callbacks.ModelCheckpoint("best.keras", save_best_only=True),
    # Cut the learning rate when progress plateaus
    keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2),
    # Log everything for TensorBoard
    keras.callbacks.TensorBoard(log_dir="./logs"),
]

model.fit(X_train, y_train, epochs=100, callbacks=callbacks,
          validation_data=(X_val, y_val))
```

KEY IDEA — EARLYSTOPPING AND WHY IT MATTERS

Train too long and the model starts **memorising** the training data instead of learning general patterns. This is **overfitting**, and its signature is unmistakable: training loss keeps falling while validation loss starts *rising*. `EarlyStopping` watches for exactly that and halts. **"Training loss down, validation loss up" is the single most important diagnostic in all of ML — know it cold.**

8.6 Overfitting, underfitting, and the fixes

Symptom	Diagnosis	Fixes
Training loss low, **validation loss high**	**Overfitting** — memorised the training set	More data · Dropout · L2 regularisation · Data augmentation · Simpler model · Early stopping
Training loss **and** validation loss both high	**Underfitting** — model too weak to learn the pattern	Bigger model · Train longer · Better features · Higher learning rate
Loss is `NaN`	Exploding gradients	Lower the learning rate · Gradient clipping · Normalise inputs
Loss doesn't move at all	Learning rate far too low, or a bug	Raise the LR · Check you called `zero_grad()` · Check the data isn't all one class

8.7 Transfer learning — what you'll actually do

You will almost never train a model from scratch. You take a model that someone else trained on millions of images or documents, and adapt it to your task. **This is transfer learning, and it is how essentially all applied ML works today.**

```
# ── KERAS ──
base = keras.applications.ResNet50(
    weights="imagenet",     # pretrained on 1.4M images
    include_top=False,      # chop off the original classification head
    input_shape=(224, 224, 3),
)
base.trainable = False      # FREEZE it — don't destroy what it learned

model = keras.Sequential([
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(1, activation="sigmoid"),   # YOUR new head: cat vs dog
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(train_data, epochs=5)

# Optional second stage: unfreeze the top layers and fine-tune at a TINY LR
base.trainable = True
for layer in base.layers[:-20]:
    layer.trainable = False
model.compile(optimizer=keras.optimizers.Adam(1e-5), loss="binary_crossentropy")
model.fit(train_data, epochs=5)
```

ANALOGY

The pretrained model already knows what edges, textures, shapes, eyes, and fur look like — it learned that from a million photos. You're not teaching it to see. You're just teaching it that *this* particular combination of fur and ears means "cat." That takes 200 examples, not a million. And the tiny learning rate in stage two matters because you want to gently *adjust* what it knows, not bulldoze it.

8.8 Framework comparison

	PyTorch	TensorFlow / Keras
Training loop	You write it — full visibility	`.fit()` — one line
Graph	Dynamic (define-by-run). Easy to debug with a plain breakpoint.	Static by default (compiled). Faster, harder to introspect.
Debugging	Just Python. `print()` and `pdb` work.	Harder inside compiled graphs.
Dominant in	Research, LLMs, anything novel	Enterprise, mobile (TF Lite), browser (TF.js)
Deployment	TorchServe, ONNX, TorchScript	TF Serving, TF Lite, TF.js — more mature
Learning curve	Steeper, but you understand more	Gentler, but more is hidden

INTERVIEW QUESTION

Q: PyTorch or TensorFlow — which would you choose, and why?

PyTorch for research and anything novel, because the dynamic graph means it's just Python — I can drop a breakpoint anywhere and inspect a tensor. Nearly all new model architectures, and virtually the entire LLM ecosystem including Hugging Face, are PyTorch-first, so I'd be swimming with the current. **TensorFlow/Keras for production deployment**, particularly on mobile via TF Lite or in a browser via TF.js, where the tooling is genuinely more mature. And **Keras specifically for rapid prototyping** of standard architectures — if I just need a CNN on tabular or image data with early stopping and checkpointing, `.fit()` with callbacks gets me there in ten lines. In practice the frameworks have converged a lot; the choice is now more about ecosystem and team familiarity than capability.

INTERVIEW QUESTION

Q: Explain backpropagation to someone non-technical.

The network makes a guess and we measure how wrong it was — that's the loss. Then we work *backwards* through the network asking, at each layer, "how much did *you* contribute to that error?" Each layer passes the blame back to the layer before it. Once every weight knows how much it contributed to the mistake, we nudge each one slightly in the direction that would have made the answer better. Repeat a few million times and the mistakes get small. **Mathematically it's just the chain rule from calculus, applied automatically.**

9. Part VIII — Hands-On Projects

Reading gets you understanding. Building gets you the job. Do these in order — each one adds exactly one new idea to the last.

> HOW TO USE THIS SECTION
>
> Type the code out — don't paste it. Then **break it deliberately**: change the chunk size and watch the answers change. Delete `zero_grad()` and watch training fail. Set `k=1` and see what gets missed. **You learn far more from watching something break in a way you predicted than from watching it work.**

Project 1 — Chat with your PDF

What you'll learn: the complete RAG pipeline, end to end. This is *the* portfolio project — nearly every AI job description implies it.

```
"""
project_1_pdf_chat.py
Ask questions about any PDF. With citations and conversation memory.

    pip install langchain langchain-openai langchain-chroma pypdf
    python project_1_pdf_chat.py mydoc.pdf
"""
import sys, os
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser
from langchain_core.messages import HumanMessage, AIMessage

PDF_PATH = sys.argv[1] if len(sys.argv) > 1 else "sample.pdf"
DB_DIR   = "./chroma_pdf"

llm        = ChatOpenAI(model="gpt-4o-mini", temperature=0)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# ── INDEXING (only if not already done) ──────────────────────
if not os.path.exists(DB_DIR):
    print(f"Indexing {PDF_PATH}...")
    docs = PyPDFLoader(PDF_PATH).load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)
    print(f"  {len(docs)} pages → {len(chunks)} chunks")
    vs = Chroma.from_documents(chunks, embeddings, persist_directory=DB_DIR)
else:
    print("Loading existing index...")
    vs = Chroma(persist_directory=DB_DIR, embedding_function=embeddings)

retriever = vs.as_retriever(search_kwargs={"k": 4})

# ── QUERY REWRITING (so follow-ups work) ─────────────────────
rewrite = ChatPromptTemplate.from_messages([
    ("system", "Rewrite the follow-up as a standalone question using the history. "
               "Output ONLY the rewritten question."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
]) | llm | StrOutputParser()

# ── ANSWERING ────────────────────────────────────────────────
answer = ChatPromptTemplate.from_template(
    "Answer using ONLY the context. If it's not there, say you don't know.\n\n"
    "Context:\n{context}\n\nQuestion: {question}\n\nAnswer:"
) | llm | StrOutputParser()

history = []
print("\nAsk me about the document. Type 'quit' to exit.\n")

while True:
    q = input("You: ").strip()
    if q.lower() in ("quit", "exit"):
        break

    # Only rewrite if there IS a history
    search_q = rewrite.invoke({"input": q, "history": history}) if history else q

    docs = retriever.invoke(search_q)
    context = "\n\n---\n\n".join(d.page_content for d in docs)
    result  = answer.invoke({"context": context, "question": q})

    pages = sorted({d.metadata.get("page", 0) + 1 for d in docs})
    print(f"\nBot: {result}")
    print(f"     [sources: pages {', '.join(map(str, pages))}]\n")

    history.extend([HumanMessage(content=q), AIMessage(content=result)])
    history = history[-6:]      # keep the last 3 exchanges
```

Extensions to attempt

- Add hybrid search with `BM25Retriever` + `EnsembleRetriever` and compare the results.
- Wrap it in a Streamlit UI: `pip install streamlit`, then `st.chat_input()` and `st.chat_message()`. This makes it demoable in an interview.
- Support multiple PDFs, and add a metadata filter so users can restrict to one document.

Project 2 — A tool-using agent

What you'll learn: LangGraph's state, nodes, conditional edges, and the loop — plus persistent memory.

```
"""
project_2_agent.py
A LangGraph agent with a calculator, a fake weather tool, and memory.

    pip install langgraph langchain-openai
"""
from typing import TypedDict, Annotated, List
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.messages import BaseMessage, HumanMessage, ToolMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
import ast, operator as op

# ── TOOLS ─────────────────────────────────────────────────
SAFE_OPS = {ast.Add: op.add, ast.Sub: op.sub, ast.Mult: op.mul,
            ast.Div: op.truediv, ast.Pow: op.pow, ast.USub: op.neg}

def _safe_eval(node):
    """Evaluate math WITHOUT eval() — eval() on model output is a security hole."""
    if isinstance(node, ast.Constant):  return node.value
    if isinstance(node, ast.BinOp):     return SAFE_OPS[type(node.op)](
                                            _safe_eval(node.left), _safe_eval(node.right))
    if isinstance(node, ast.UnaryOp):   return SAFE_OPS[type(node.op)](_safe_eval(node.operand))
    raise ValueError("unsupported expression")

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression. Use this for ALL arithmetic — never compute mentally.

    Args:
        expression: e.g. '4271 * 89' or '(15 + 3) / 2'
    """
    try:
        return str(_safe_eval(ast.parse(expression, mode="eval").body))
    except Exception as e:
        return f"Error: {e}"

@tool
def get_weather(city: str) -> str:
    """Get today's weather for a city.

    Args:
        city: e.g. 'Lahore'
    """
    fake = {"lahore": "34°C, sunny", "london": "12°C, raining", "tokyo": "19°C, cloudy"}
    return fake.get(city.lower(), f"No data for {city}")

tools = [calculator, get_weather]
registry = {t.name: t for t in tools}
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)

# ── STATE ─────────────────────────────────────────────────
class State(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

# ── NODES ─────────────────────────────────────────────────
def agent(state: State):
    return {"messages": [llm.invoke(state["messages"])]}

def tool_executor(state: State):
```

```
    out = []
    for call in state["messages"][-1].tool_calls:
        print(f"    [calling {call['name']}({call['args']})]")
        result = registry[call["name"]].invoke(call["args"])
        out.append(ToolMessage(content=str(result), tool_call_id=call["id"]))
    return {"messages": out}

def route(state: State):
    return "tools" if state["messages"][-1].tool_calls else END

# ── GRAPH ──────────────────────────────────────────────
b = StateGraph(State)
b.add_node("agent", agent)
b.add_node("tools", tool_executor)
b.add_edge(START, "agent")
b.add_conditional_edges("agent", route)
b.add_edge("tools", "agent")            # ← the loop
graph = b.compile(checkpointer=MemorySaver())

# ── RUN ────────────────────────────────────────────────
cfg = {"configurable": {"thread_id": "demo"}, "recursion_limit": 10}
print("Agent ready. Type 'quit' to exit.\n")

while True:
    q = input("You: ").strip()
    if q.lower() in ("quit", "exit"):
        break
    result = graph.invoke({"messages": [HumanMessage(content=q)]}, cfg)
    print(f"Bot: {result['messages'][-1].content}\n")

# Try: "What is 4271 * 89?"
# Then: "What's the weather in Lahore and in Tokyo?"
# Then: "What did I ask you first?"   ← memory works, thanks to the checkpointer
```

Extensions to attempt

- Add `interrupt_before=["tools"]` and build an approval prompt. Now you have human-in-the-loop.
- Add an `iterations: int` field to the state and force `END` after 5 loops. Now you have a guardrail.
- Give the agent a real tool — the Tavily search API has a free tier.

Project 3 — Self-correcting RAG (Corrective RAG)

What you'll learn: combining RAG with LangGraph. This is the project that separates you from candidates who only did the tutorial. The agent *grades its own retrieved documents* and re-searches if they're bad.

```
"""
project_3_crag.py — RAG that checks its own work.

Flow:  retrieve → grade each doc → if docs are bad, rewrite the query
       and retrieve again → if still bad, fall back to web search → generate
"""
from typing import TypedDict, List
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, START, END

class GradeDocs(BaseModel):
    """Binary relevance score for a retrieved document."""
    relevant: bool = Field(description="True if the document helps answer the question")

grader = llm.with_structured_output(GradeDocs)

class CRAGState(TypedDict):
    question: str
    documents: List[str]
    generation: str
    attempts: int

def retrieve(state):
    docs = retriever.invoke(state["question"])
    return {"documents": [d.page_content for d in docs],
```

```
            "attempts": state.get("attempts", 0) + 1}

def grade_documents(state):
    """Keep only the docs an LLM judges relevant."""
    kept = []
    for d in state["documents"]:
        verdict = grader.invoke(
            f"Question: {state['question']}\n\nDocument: {d}\n\nIs it relevant?"
        )
        if verdict.relevant:
            kept.append(d)
    print(f"  graded: kept {len(kept)}/{len(state['documents'])}")
    return {"documents": kept}

def rewrite_query(state):
    better = (llm | StrOutputParser()).invoke(
        f"Rewrite this to be a better search query: {state['question']}"
    )
    print(f"  rewrote query → {better}")
    return {"question": better}

def generate(state):
    ctx = "\n\n".join(state["documents"]) or "(no relevant documents found)"
    ans = answer_chain.invoke({"context": ctx, "question": state["question"]})
    return {"generation": ans}

# ── THE ROUTER: this is where the intelligence lives ─────────
def decide(state):
    if state["documents"]:
        return "generate"                    # we have good docs → answer
    if state["attempts"] >= 3:
        return "generate"                    # give up gracefully — GUARDRAIL
    return "rewrite"                         # bad docs → try a better query

b = StateGraph(CRAGState)
b.add_node("retrieve", retrieve)
b.add_node("grade", grade_documents)
b.add_node("rewrite", rewrite_query)
b.add_node("generate", generate)

b.add_edge(START, "retrieve")
b.add_edge("retrieve", "grade")
b.add_conditional_edges("grade", decide, {"generate": "generate", "rewrite": "rewrite"})
b.add_edge("rewrite", "retrieve")            # ← THE LOOP
b.add_edge("generate", END)

crag = b.compile()
print(crag.invoke({"question": "What is the parental leave policy?"})["generation"])
```

WHY THIS PROJECT MATTERS FOR YOUR INTERVIEW

It demonstrates the exact thing LangGraph exists for. The number of loops is **decided at runtime** by an LLM's judgement of its own retrieval quality. **You cannot express this as a chain.** If you can explain this project, you can answer "why LangGraph over LangChain?" with a real example instead of a definition.

Project 4 — Image classifier, in both frameworks

What you'll learn: the training loop, and the concrete difference between PyTorch and Keras.

```
"""
project_4_mnist.py — the same model, twice.
Handwritten digit classification. The 'hello world' of deep learning.
"""

# ═══════════════ KERAS VERSION ═══════════════
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

(X_tr, y_tr), (X_te, y_te) = keras.datasets.mnist.load_data()
X_tr, X_te = X_tr / 255.0, X_te / 255.0     # normalise to 0-1. ALWAYS do this.
```

```
model = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),                       # 28x28 image → 784-long vector
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(X_tr, y_tr, epochs=5, validation_split=0.1,
          callbacks=[keras.callbacks.EarlyStopping(patience=2,
                                                   restore_best_weights=True)])
print("Keras test acc:", model.evaluate(X_te, y_te)[1])     # ~0.978


# ═══════════════════ PYTORCH VERSION ═══════════════════
import torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

tfm = transforms.Compose([transforms.ToTensor(),
                          transforms.Normalize((0.1307,), (0.3081,))])
train = DataLoader(datasets.MNIST("./d", train=True,  download=True, transform=tfm),
                   batch_size=64, shuffle=True)
test  = DataLoader(datasets.MNIST("./d", train=False, download=True, transform=tfm),
                   batch_size=1000)

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(), nn.Linear(784, 128), nn.ReLU(),
            nn.Dropout(0.2), nn.Linear(128, 10),
        )
    def forward(self, x):
        return self.net(x)

dev = "cuda" if torch.cuda.is_available() else "cpu"
model = Net().to(dev)
crit  = nn.CrossEntropyLoss()
opt   = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    model.train()
    for X, y in train:
        X, y = X.to(dev), y.to(dev)
        opt.zero_grad()               # 1
        loss = crit(model(X), y)      # 2 + 3
        loss.backward()               # 4
        opt.step()                    # 5
    print(f"epoch {epoch+1} done")

model.eval()
correct = 0
with torch.no_grad():
    for X, y in test:
        X, y = X.to(dev), y.to(dev)
        correct += (model(X).argmax(1) == y).sum().item()
print(f"PyTorch test acc: {correct / len(test.dataset):.4f}")   # ~0.978
```

Same architecture. Same accuracy. Keras: 12 lines. PyTorch: 25 lines. Now you can speak to the trade-off from experience, not from a blog post.

Project 5 — Capstone: the full-stack agent

Combine everything. This is the one you put on your CV.

SPEC

A research assistant that answers questions about your documents AND the live web.

- **LangGraph** agent with a supervisor node that routes each query.
- **Tool 1:** a RAG retriever over your indexed PDFs (built with **LlamaIndex**, wrapped as a LangChain tool).
- **Tool 2:** a live web search (Tavily).
- **Tool 3:** a calculator.
- **ChromaDB** for the vector store (swap to Pinecone to prove you can).
- **Checkpointer** for conversation memory across sessions.
- **Human-in-the-loop** approval before any web search (to demonstrate the pattern).
- **Streamlit** UI showing the agent's reasoning steps live.
- Every answer carries **citations**.

When an interviewer asks "tell me about a project you've built," this is a five-minute answer that touches every single item on the job description. Deploy it (Streamlit Community Cloud is free) and put the link on your CV.

10. Part IX — Interview Preparation

10.1 The questions you will be asked

Foundations

1. **What is an embedding?** A dense vector where semantic similarity becomes geometric proximity. It lets you find text that *means* the same thing, not just text containing the same words.
2. **What is a context window?** The max tokens a model can process at once — prompt *and* response combined. It's the constraint that forces RAG to exist.
3. **What is temperature?** Controls randomness in token selection. Use `0` for anything factual; higher for creative work.
4. **Why do LLMs hallucinate?** They're trained to produce plausible text, not true text. They have no mechanism to distinguish "I know this" from "this pattern seems likely." There is no internal confidence signal being surfaced.
5. **RAG vs fine-tuning?** RAG adds *knowledge* (facts, up-to-date, citable, cheap to update). Fine-tuning changes *behaviour* (tone, format, reasoning style). Most people who ask for fine-tuning need RAG.

LangChain

6. **What is LCEL and why does the pipe work?** Every component implements the `Runnable` interface (`invoke` / `batch` / `stream` /async). Because they share an interface, they compose with `|`. And because a chain is itself a Runnable, you get parallelism, streaming, and async on any chain for free.
7. **Chain vs agent?** Chain = fixed path you designed. Agent = path the model decides at runtime.
8. **How does tool calling work?** The model doesn't execute anything — it *requests* a call with structured arguments. *Your code* executes it and feeds the result back. This is a security point as much as a mechanical one.
9. **How does memory work?** It doesn't, natively — you re-send the conversation each time. "Memory" is a strategy for managing that list: buffer (keep all), window (keep last k), or summary (compress the old).
10. **How do you get reliable structured output?** `with_structured_output(PydanticModel)`, which uses the provider's native function-calling API. Never prompt for JSON and hope.

RAG

11. **Walk me through a RAG pipeline.** Indexing: load → split → embed → store. Querying: embed question → similarity search → stuff top-k into prompt → generate.
12. **How do you pick a chunk size?** Trade-off: small = precise retrieval but fragmented context; large = complete context but fuzzy retrieval. Depends on content structure. It's empirical — you test it.
13. **What's hybrid search and why?** Vector + BM25 keyword. Vector search is bad at exact strings (product codes, names, acronyms) because they carry little semantic signal. Hybrid covers both.
14. **What is reranking?** Retrieve 25 cheaply, then use a slower cross-encoder to reorder and keep the best 4. Better accuracy *and* fewer tokens to the expensive model. Highest-ROI RAG upgrade.
15. **How do you evaluate RAG?** Separately. Retrieval: hit rate, MRR. Generation: faithfulness, answer relevance, context precision (RAGAS). An end-to-end number tells you nothing actionable.
16. **My RAG gives bad answers. Debug it.** Print the retrieved chunks. If the right chunk isn't there → retrieval problem. If it *is* there → generation problem. Fix accordingly. (See §4.5.)

LangGraph

17. **Why LangGraph over LangChain?** Cycles (chains are DAGs), explicit inspectable state, real control via conditional edges, and persistence via checkpointers (memory, pause/resume, human-in-the-loop, time travel).
18. **What is a reducer?** The function that decides *how* a node's returned value merges into state. Default = overwrite. `add_messages` / `operator.add` = append. Forgetting it on the messages field is the classic bug.

19. **What is a conditional edge?** A function that reads state and returns the name of the next node. It's the agent's `if` statement, and it's what creates the loop.
20. **What does a checkpointer give you?** Persistence, pause/resume, human-in-the-loop approval, and time-travel debugging — because state is one explicit object that can be snapshotted after every node.
21. **How do you stop an agent looping forever?** A `recursion_limit`, plus an iteration counter in state that the conditional edge checks. *Mention this unprompted — it signals production thinking.*

Vector DBs & LlamaIndex

22. **Why not just use Postgres?** You can — pgvector works. But dedicated vector DBs have purpose-built ANN indexes (HNSW), which is the difference between milliseconds and seconds at scale.
23. **What is HNSW?** The dominant ANN algorithm. A multi-layer graph: coarse layers for fast long jumps, fine layers for precise local search. Trades a little recall for enormous speed.
24. **Chroma vs Pinecone vs Weaviate?** Chroma = embedded, free, prototypes. Pinecone = managed, scales, no ops burden. Weaviate = self-hostable, best-in-class hybrid search.
25. **LangChain vs LlamaIndex?** LangChain = general orchestration framework. LlamaIndex = specialised for ingestion and retrieval. They interoperate — a common pattern is LlamaIndex retrieval wrapped as a tool inside a LangGraph agent.

ML frameworks

26. **Explain backpropagation.** Forward pass computes a prediction and a loss. Backward pass walks the computation graph in reverse, applying the chain rule to find each weight's contribution to the error. The optimiser then nudges each weight to reduce it.
27. **Why do you need an activation function?** Without non-linearity, stacked linear layers collapse into a single linear layer — a 50-layer net would be no more powerful than a 1-layer one. ReLU is what lets networks learn curved decision boundaries.
28. **What is overfitting and how do you fix it?** The model memorises training data instead of learning patterns. **Signature: training loss falls while validation loss rises.** Fix with more data, dropout, L2, augmentation, a simpler model, or early stopping.
29. **PyTorch vs TensorFlow?** PyTorch: dynamic graph, easy debugging, dominant in research and the entire LLM ecosystem. TF/Keras: mature deployment (Lite, JS, Serving), `.fit()` for rapid prototyping.
30. **What is transfer learning?** Take a model pretrained on a huge dataset, freeze most of it, and train a small new head on your task. Needs hundreds of examples instead of millions. It's how virtually all applied ML actually works.

10.2 Cheat sheet

Concept	The one-line answer
Embedding	Meaning → geometry. Similar text, nearby vectors.
RAG	Don't send the book. Send the three relevant paragraphs.
LCEL / Runnable	One shared interface → components compose with `|` → free batching, streaming, async.
Tool calling	The model *requests*. Your code *executes*.
Agent	A loop: model → tools → model → … until done.
LangGraph	Chains are DAGs and can't loop. Graphs can.
Reducer	How a node's update merges into state. Append vs overwrite.
Conditional edge	The agent's `if`. Reads state, returns the next node's name.
Checkpointer	Snapshot state after every node → memory, resume, approval, time travel.
Vector DB	A database indexed for nearest-neighbour search, not exact match.
Hybrid search	Vector finds meaning. BM25 finds exact strings. You need both.
Reranking	Retrieve 25 cheap, rerank, keep 4. Best accuracy-per-token upgrade there is.
Backprop	The chain rule, applied automatically, backwards through the graph.
Overfitting	Train loss ↓, validation loss ↑. Memorising, not learning.
Transfer learning	Freeze someone else's model. Train a small new head.

10.3 Glossary

Agent	An LLM in a loop with tools, deciding its own next action.
ANN	Approximate Nearest Neighbour. Fast, slightly-inexact vector search.
Autograd	Automatic differentiation. Computes gradients so you don't do calculus by hand.
Backpropagation	Walking the computation graph backwards to compute each weight's gradient.
Batch	A group of samples processed together in one forward/backward pass.
BM25	A classic keyword-ranking algorithm. The "sparse" half of hybrid search.
Chain	A fixed, linear sequence of LangChain components joined with `|`.
Checkpointer	LangGraph component that persists state after every node.
Chunk	A piece of a document, sized to be embedded and retrieved.
Context window	Max tokens the model can see at once. Prompt + response.
Cosine similarity	Angle between two vectors. The standard text-similarity metric.
DAG	Directed Acyclic Graph. No cycles — which is exactly why chains can't loop.
Dropout	Randomly zeroing neurons during training to prevent overfitting.
Embedding	A vector representing the meaning of text.
Epoch	One complete pass through the training data.
Few-shot	Giving the model examples in the prompt rather than describing the task.

Fine-tuning	Updating a model's actual weights on your data.
Gradient	The derivative of loss w.r.t. a weight. "Which way, and how much."
Hallucination	Confidently-stated false output.
HNSW	Hierarchical Navigable Small World. The dominant ANN index.
Hybrid search	Vector search + keyword search, combined.
LCEL	LangChain Expression Language. The `|` composition syntax.
Learning rate	Step size in gradient descent. The most important hyperparameter.
Loss	A single number measuring how wrong the model's prediction was.
Node	LangGraph: a Python function that takes state and returns an update.
Optimiser	The algorithm that applies gradients to weights. Adam is the default.
Overfitting	Memorising training data. Train loss down, validation loss up.
Prompt template	A reusable prompt with `{placeholders}`.
RAG	Retrieve relevant text, put it in the prompt, generate an answer from it.
Reducer	LangGraph: how a node's returned value merges into existing state.
ReLU	`max(0, x)`. The standard activation function.
Reranker	A model that reorders retrieved docs by true relevance.
Retriever	Anything that takes a query and returns relevant documents.
Runnable	The LangChain interface every component implements. The reason `|` works.
State	LangGraph: the shared dict flowing through the graph.
Temperature	Randomness knob. `0` = deterministic.
Tensor	A multi-dim array that can live on a GPU and track gradients.
Token	~3-4 characters. The unit an LLM reads, writes, and bills in.
Tool calling	The model requesting a function call with structured arguments.
Transfer learning	Reusing a pretrained model, retraining only a small head.
Vector DB	A database with an index built for nearest-neighbour search.

10.4 Your two-week plan

Days	Do this
1–2	Read Parts I–II. Run every code snippet. Build a `prompt | llm | parser` chain and a `with_structured_output` extractor.
3–4	Read Part III. **Build Project 1.** Then deliberately break it: set `k=1`, set chunk size to 100, remove "only use the context." Watch what each change does.
5–6	Read Part IV. **Build Project 2.** Write the graph by hand — do not use `create_react_agent`. Add human-in-the-loop.
7	Read Part V and VI. Swap Project 1's Chroma for Pinecone. Rebuild Project 1 in LlamaIndex in 5 lines and compare.
8–9	**Build Project 3.** This is the one that makes your "why LangGraph" answer concrete.
10–11	Read Part VII. **Build Project 4** in both frameworks. Delete `zero_grad()` and watch training fail.
12–13	**Build Project 5.** Deploy it. Put the link on your CV.
14	Answer all 30 questions in §10.1 **out loud**, without notes. Record yourself. The gap between knowing and articulating is real — close it before the interview, not during.

FINAL ADVICE

Interviewers do not want to hear definitions — they can read the docs too. They want to hear **trade-offs** and **war stories**. "I used chunk size 1000 with 200 overlap, but retrieval was missing answers that spanned sections, so I added a reranker and pulled 20 candidates instead of 4" is worth more than any textbook definition. **Build the projects. Break them. Then you'll have the stories.**

STUDY COMPANION · CONTINUES AFTER PART IX

The Production & MLOps Companion

Extending *The Beginner's Guide to Building AI Applications* — Parts X to XVII

This document picks up exactly where the guide leaves off. The guide is excellent on the *AI* half of an AI/ML role — LLMs, embeddings, RAG, LangChain, LangGraph, vector databases, LlamaIndex, and the ML frameworks. It is deliberately quiet on the *engineering* half: how a model actually gets deployed, the MLOps toolchain, the shape of a production system, and the full-stack web development the job description also asks for.

Those are the topics below. They are written in the same voice as the guide — with the same four callout boxes:

> **ANALOGY**
>
> — a hard idea compared to something familiar. **KEY IDEA** — the one sentence to remember. **COMMON MISTAKE** — a trap that costs beginners hours or interview points. **INTERVIEW QUESTION** — a likely question with a strong answer.

Read this the same way you read the guide: understand the concepts, and know the trade-offs cold. Interviewers on this kind of role probe the deployment and architecture side hard, because it's where "I've read about it" and "I've shipped it" separate most visibly.

Part X — From Notebook to Production: What "Deployment" Actually Means

Everything in the original guide runs in a script or a notebook — you call `.invoke()` and read the answer. Production is different: real users, over the network, at the same time, reliably, without you sitting there running cells.

Deploying a model means putting it behind an *endpoint* — a URL that accepts a request (an input) and returns a response (a prediction), over HTTP, that any application can call.

KEY IDEA

"Deploy a model" = "wrap it in a service with a URL." Everything else in MLOps — GPUs, autoscaling, load balancing, containers — is machinery for making that endpoint *fast, reliable, and affordable*. If you hold onto that one sentence, every tool below has an obvious job.

ANALOGY A trained model sitting in a file is like a chef who knows how to cook but has no restaurant. Deployment is opening the restaurant: a front door people can walk through (the endpoint), a kitchen that can handle many orders at once (scaling), and a manager watching that nothing catches fire (monitoring).

The three ways to deploy a model

This is the single most useful mental model for the whole topic, and a very likely interview question. There are three broad paths, and choosing between them is mostly about one question: *do you need to run your own model, and does your data have to stay in-house?*

Path	What you manage	GPU?	Best when
Hosted model API (OpenAI, Anthropic, Cohere, AWS Bedrock)	Nothing — you call someone else's endpoint	No	Fastest path. You want a capable model *now* and are fine with per-request pricing and your data leaving your environment.
Managed platform (SageMaker, Vertex AI)	Your model; the platform provisions GPUs, endpoint, scaling	Yes (managed)	You need *your own* model (e.g. a fine-tuned Llama) or must keep data in your cloud, without running raw servers.
Self-managed infra (vLLM / TGI / Triton on Kubernetes)	Everything — machines, serving stack, scaling, uptime	Yes (you rent it)	Maximum control, best cost at very large scale, or strict isolation. You have (or are) an infra team.

COMMON MISTAKE

Assuming "deploying an LLM" always means renting GPUs and standing up SageMaker. A large share of production RAG apps never deploy a model at all — they call a **hosted API** for generation and only *host their own embedding model* (which is small and often runs on CPU). Reaching for the heavy path first is the classic over-engineering error.

Where you actually need a GPU

"We need a GPU" is true for *one specific thing* — running a large generation model's inference — not for the whole system.

- **Large LLM inference** → GPU (unless you use a hosted API, in which case the provider's GPU).
- **Embedding model** → usually fine on CPU; it's a much smaller model.
- **The API/orchestration layer, the vector DB, the web frontend/backend** → CPU. None of these touch a GPU.

> KEY IDEA
>
> Match the hardware to the component, not to the project. Most of a RAG system is ordinary CPU software; the GPU is needed only where a big neural network runs a forward pass. This is how you avoid paying for GPUs you don't need.
>
> **INTERVIEW QUESTION Q:** How would you deploy an LLM-powered feature? Walk me through the options. **A:** It depends on constraints. If we can use a third-party model and data residency isn't an issue, a hosted API (OpenAI/Anthropic/Bedrock) is fastest — no infra, pay per token. If we need our own or fine-tuned model, or data must stay in our cloud, a managed platform like SageMaker or Vertex AI provisions the GPU endpoint and handles scaling for us. If we're at very large scale or need tight control and want to avoid per-token costs, we self-host an open-weight model with a serving stack like vLLM or TGI on Kubernetes. I'd start with the simplest path that meets the constraints and only move down the list when scale or policy forces it — deploying your own GPU fleet before you've validated the product is premature.

Part XI — The MLOps Toolchain

The job description lists a pile of tools — Docker, Kubernetes, SageMaker, Vertex AI, MLflow, Kubeflow, CI/CD — under the general banner of "deployment." They are *not* interchangeable; they're layers and specializations. Here's each one, in the order a model actually flows through them.

Docker — packaging (containerization)

A **container** is a lightweight, self-contained box holding your model *plus everything it needs to run*: the code, the exact library versions, system dependencies. Docker freezes that whole environment into an **image**, and the image runs identically everywhere — your laptop, a teammate's machine, a cloud server.

- **The problem it solves:** "it works on my machine but breaks in production," which happens because environments differ.

- **How it works:** you write a small recipe file (a `Dockerfile`) listing what to install and how to start your app → Docker builds it into an image → the image runs as a container anywhere Docker is installed.
- **Its role in deployment:** you package your model-serving app into a container, and now it's a *portable unit* you can ship anywhere. Almost everything below builds on this.

> **KEY IDEA**
>
> The container is the atomic unit of modern deployment. When someone says "we containerized the model," they mean "we packaged it so it runs the same everywhere." Kubernetes, SageMaker, and CI/CD all operate on containers.

Kubernetes — orchestration

One container is easy to run by hand. Production means *many* containers: multiple copies of your model for traffic, restarting crashed ones, adding more under load, spreading them across a fleet of machines. Doing that manually is impossible at scale.

Kubernetes is the system that manages containers for you. You declare a *desired state* — "keep 5 copies running, scale up if the GPU gets busy" — and it continuously makes reality match: it schedules containers onto machines, restarts failed ones, load-balances requests, and autoscales up and down.

- **Its role in deployment:** it turns "a container that runs my model" into "a reliable, self-healing, scalable service."
- **Why it's often hidden:** it's powerful but complex, which is exactly why the managed platforms below wrap it so you don't touch it directly.

> **ANALOGY**
>
> Docker builds one shipping container. Kubernetes is the port — the cranes, the scheduling, the logistics that move thousands of containers around and replace any that fall in the water, without a human directing each one.

SageMaker (AWS) and Vertex AI (GCP) — managed MLOps platforms

These are the all-in-one clouds for the ML lifecycle, and they're the "managed platform" path from Part X. Instead of assembling Docker + Kubernetes + monitoring + scaling yourself, you hand the model to the platform and it does the heavy lifting.

- **How deployment works on them:** you give the platform your trained model and specify the hardware → it provisions the GPU machines and stands up an HTTPS **endpoint** your app calls. Under the hood it's containers and orchestration, but you think in terms of "models" and "endpoints," not pods and clusters.
- **SageMaker** is the AWS version; **Vertex AI** is Google's near-equivalent. The JD lists both because a candidate might work in either cloud.

- They do far more than serving — training, tuning, pipelines, a model registry — but *deployment* is the piece most people mean.

> **COMMON MISTAKE**
>
> Treating SageMaker/Vertex as *only* a serving tool. They overlap with the tools below — SageMaker can do experiment tracking (overlapping MLflow) and pipelines (overlapping Kubeflow). A team fully on AWS often uses SageMaker for most of the lifecycle and skips the standalone tools entirely. The tools overlap on purpose.

MLflow — experiment tracking and model registry

This one is different: it's less about *running* the model and more about *managing which model* you deploy. Training produces many model versions with different settings and different accuracy.

- **What it does:** logs each experiment's parameters and results so you can compare runs (Tracking), and stores trained models with versions and stages like "staging" vs "production" (the Model Registry).
- **Its role in deployment:** it's the *source of truth* for what gets deployed. You promote a specific tracked version to production; MLflow can also package and serve it.

> **KEY IDEA**
>
> MLflow answers "which exact model is live, and how did it perform?" — critical for the reproducibility and production monitoring the JD emphasizes. It's the lab notebook plus the version-controlled shelf for your models.

Kubeflow — ML pipelines on Kubernetes

Kubeflow is a toolkit for running full ML *workflows* on top of Kubernetes. Recall the training pipeline: load → preprocess → train → evaluate → deploy.

- **What it does:** lets you define that whole multi-step pipeline as an automated, repeatable workflow, running each step as a container on a Kubernetes cluster, so steps scale and the whole thing re-runs on new data. Its serving component (**KServe**) puts a trained model behind an endpoint.
- **Its role:** the "do MLOps natively on Kubernetes" option — more control and cloud-agnostic (any Kubernetes, not locked to one cloud), but heavier to operate than a managed platform. It's the self-hosted alternative to SageMaker/Vertex.

CI/CD (and CT) — automation

CI/CD — continuous integration / continuous delivery — is the automation that takes a change from your code repository and safely pushes it to production without manual steps.

- **In normal software:** commit code → the pipeline automatically builds it, runs tests, deploys it.
- **In ML:** the same idea, extended to models. When a model is retrained or code changes → build the container → run tests *and evaluation* → deploy the new version. The extra "**CT**" you'll sometimes see is *continuous training* — automatically retraining on fresh data.
- **Common tools:** GitHub Actions, GitLab CI, Jenkins.
- **Its role:** makes releases *repeatable and safe* instead of a risky manual push. This is what separates a hobby project from a system a team updates reliably many times a week.

The master table

Tool	Category	What it does	How it works (one line)
Docker	Packaging	Bundles model + code + dependencies into a portable container	Write a `Dockerfile`; it builds an image that runs identically everywhere
Kubernetes	Orchestration	Runs, scales, and self-heals many containers across machines	Declare desired state; it continuously makes reality match
SageMaker	Managed MLOps (AWS)	Full lifecycle on AWS — train, deploy, serve, monitor	Hand it a model; it provisions GPUs and exposes an endpoint
Vertex AI	Managed MLOps (GCP)	Google's equivalent of SageMaker	Same idea, on Google Cloud
MLflow	Tracking & registry	Tracks experiments, versions models, marks "production"	Logs each run's params/results; stores and promotes model versions
Kubeflow	ML pipelines on K8s	Defines full ML workflows as automated pipelines	Each step runs as a container on a Kubernetes cluster (KServe for serving)
CI/CD (GitHub Actions, GitLab CI, Jenkins)	Automation	Auto-builds, tests, and deploys on every change	A commit triggers build → test → deploy without manual steps

How they fit together

Follow one model from "trained" to "answering live users":

MLflow tracks and selects the model version → Docker packages it into a container → a CI/CD pipeline automatically builds, tests, and ships that container → it runs on Kubernetes (or a managed platform like SageMaker / Vertex AI) which serves it behind an endpoint, scales it, and keeps it healthy → Kubeflow can wrap the earlier training-to-deploy steps into one automated pipeline.

> KEY IDEA
>
> These are layers and specializations, not competitors. A real team rarely uses *all* of them — they pick a combination that fits their cloud and scale. On AWS: often Docker + SageMaker + a CI/CD tool, and that's it. Naming the *right subset* for a given situation is a stronger interview signal than listing every tool.
>
> **COMMON MISTAKE** Also worth knowing: there's a serving-framework layer *inside* the box that runs the model. **vLLM** and **TGI** (Text Generation Inference) are optimized servers for LLMs specifically; **Triton**, **TorchServe**, and **TF Serving** serve models generally; **BentoML** and **Ray Serve** are Python-native serving frameworks. You don't need all of them, but if asked "how do you actually serve an open-weight LLM efficiently," the answer "vLLM or TGI in a container" is the one interviewers are listening for.

Part XII — Pipelines & the ML Lifecycle

Two different things both get called "pipeline," and conflating them is the single most common MLOps mix-up.

ML pipeline vs deployment pipeline

- An **ML pipeline** is the sequence of *ML work* that produces a trained model: collect data → preprocess → train → evaluate.
- A **deployment pipeline** (really a CI/CD pipeline) is the *automation* that takes a finished model and ships it to production reliably: build → test → deploy → monitor.

One *produces* the model; the other *delivers* it. They connect — deployment is usually the last stage of the broader ML pipeline, or a separate pipeline triggered once a model is ready — but they are **not the same thing**.

> **KEY IDEA**
>
> A typical ML pipeline *includes* deployment as its final stage, but the ML pipeline as a whole is much bigger than deployment — most of it is the data-and-training work that happens *before* anything ships. "Deployment pipeline" refers specifically to the automated delivery part.

The full ML lifecycle — the six stages

1. **Data collection** — gather the raw material: databases, APIs, files, logs, or (for RAG) your documents. "Garbage in, garbage out" starts here.
2. **Preprocessing & feature engineering** — clean the data (missing values, duplicates, errors) and transform it into a form the model can learn from. *In a RAG/LLM context, this is where chunking and embedding live.*
3. **Training** — feed the prepared data to the algorithm so it learns patterns. For LLMs this is often *fine-tuning* a pretrained model rather than training from scratch. The compute-heavy, GPU-hungry stage.
4. **Evaluation** — test the trained model on data it never saw, measuring real performance (accuracy, or for RAG, answer quality). This is the *gate*: a model that fails here goes back, it doesn't ship.
5. **Deployment** — put the model that passed evaluation behind an endpoint. This is where the entire Part XI toolchain comes in. This one stage *is* the "deployment pipeline."
6. **Monitoring** — once live, watch it continuously: latency, errors, and crucially *model quality* — is it drifting, degrading, returning bad outputs?

The feedback loop is what makes this a *lifecycle* rather than a straight line: monitoring detects decay or a shift in the world (data drift) → triggers **retraining** on fresh data → back to the top. This circles indefinitely.

> **COMMON MISTAKE**
>
> Skipping stage 6. Monitoring is the stage people most often omit and most often regret. And in RAG specifically, systems quietly rot not because the model breaks but because *retrieval starts returning irrelevant chunks* — so your monitoring has to watch retrieval quality, not just uptime.

Mapping RAG onto the lifecycle

Everything in the original guide's RAG chapter slots into these six stages:

- **Data collection** = your PDFs / documents (the loaders in guide §3.8).
- **Preprocessing** = chunking + embedding (guide §4.2 indexing pipeline).

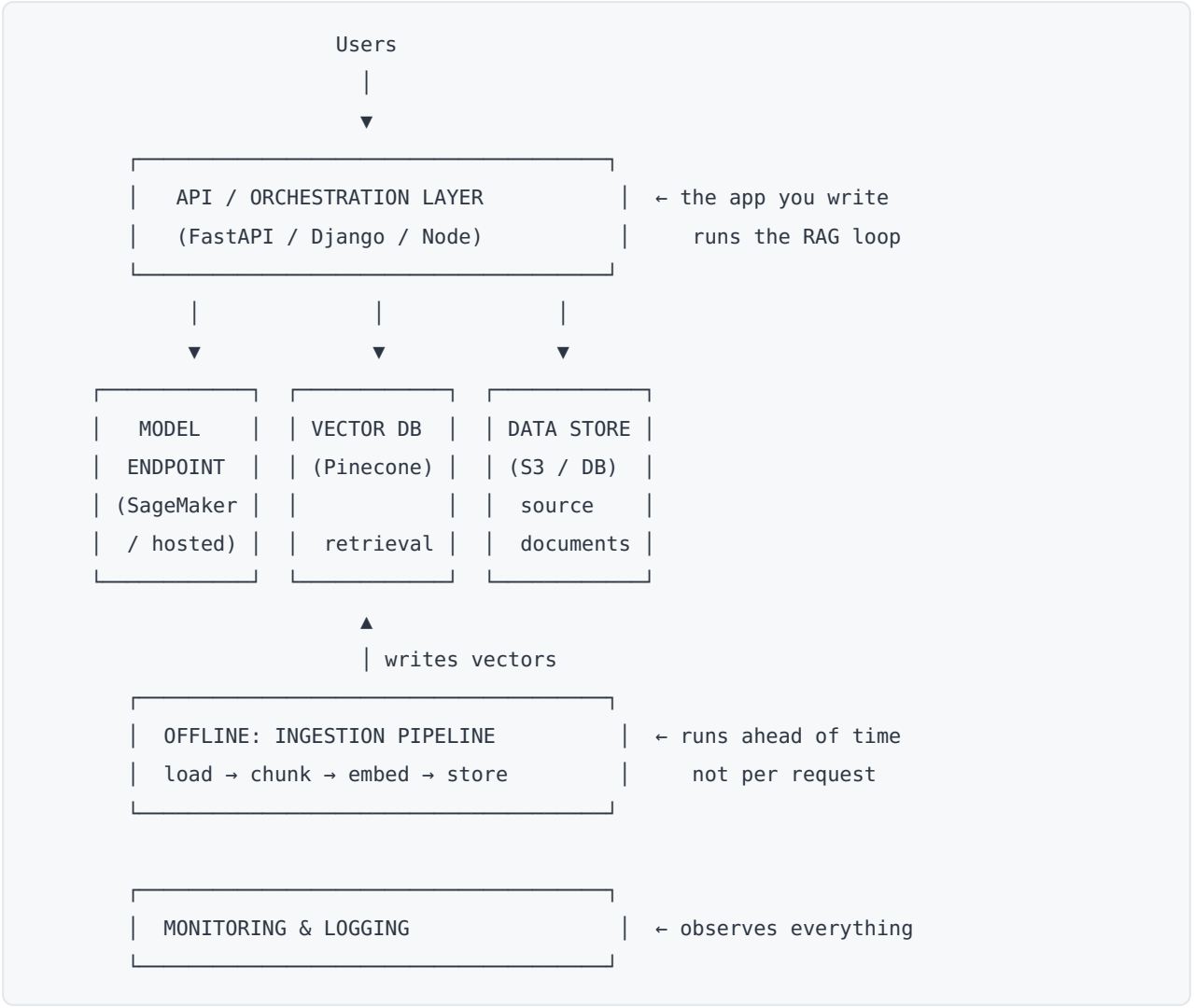
- **Training** = usually *none* for classic RAG — you use a pretrained embedding model and a pretrained/hosted LLM. (This is why RAG is cheaper than fine-tuning: you skip the expensive stage.)
- **Evaluation** = RAGAS metrics — faithfulness, answer relevance, context precision (guide §4.5).
- **Deployment** = serving the RAG chain behind an API endpoint.
- **Monitoring** = watching retrieval hit rate and answer quality live, plus cost and latency.

> **KEY IDEA**
>
> The tidy six-stage diagram is the widely-taught mental model, not a rigid law. Real pipelines loop constantly *within* the early stages — evaluation sends you back to tweak features or chunk size long before anything reaches deployment — and teams draw the boundaries slightly differently. Treat it as the map, knowing the real terrain has more back-and-forth.

Part XIII — Production RAG Architecture

The guide shows you a RAG *chain*. Production needs a RAG *system*. The difference is a few components the guide never names because they're engineering, not AI.

The full stack



The two components most beginners miss:

The API / orchestration layer — the app you write that sits in the middle and runs the RAG loop. This is where LangChain or LlamaIndex lives. When a request arrives, *this* layer embeds the query, calls the vector DB, fills the prompt template, calls the model endpoint, and returns the answer.

> **KEY IDEA**
>
> Pinecone and SageMaker don't talk to each other directly — your orchestration code is the conductor. Without it you have instruments but no one playing them. It's easy to forget because it's *code you write*, not a service you sign up for — which is exactly why it doesn't show up when you plan by listing vendors.

The ingestion pipeline — the offline job that loads documents, chunks them, embeds them, and writes vectors into the DB. It's the chunking step, run as a *job* rather than

inline. It doesn't run on every user request — it runs ahead of time, and again whenever your data changes. Nothing is *in* the vector DB until this job puts it there.

How vectors are stored: pgvector vs ChromaDB

The guide mentions pgvector in one line ("Why not just use Postgres? pgvector works"). Here's the actual storage difference, because it's a clean way to understand what a vector DB *is*.

- **pgvector** adds a `vector` **column type** to an ordinary PostgreSQL table. Your embeddings live *right next to your relational data* — an `items` table might have `id`, `content`, and an `embedding vector(1536)` column. You query with a SQL operator: `ORDER BY embedding <-> $1` finds nearest neighbours. It builds ANN indexes (`ivfflat` or `hnsw`) on that column.
- **ChromaDB** is a *purpose-built* store that organizes data into **collections**. Each item bundles four things together: an `id`, the `document` (raw text), the `embedding` (vector), and `metadata`. You query with a Python API: `collection.query(query_texts=[...], n_results=k)`.

	pgvector	ChromaDB
What it is	A vector *column* in a Postgres table	A dedicated vector store with *collections*
Data sits	Next to your relational data	Bundled: text + vector + metadata together
Query with	SQL (`<->` operator)	Python API (`.query()`)
Reach for it when	You already run Postgres and want vectors beside your normal data	You want a fast, simple store for a RAG/search prototype

> KEY IDEA
>
> "Vector DB" and "regular DB" aren't opposites — pgvector *is* a regular DB with a vector index bolted on. The real distinction is that a vector database's *primary* index is built for fast approximate-nearest-neighbour search (HNSW) in high-dimensional space, which is the difference between milliseconds and seconds at scale. (This is guide §6.1 — the addendum just shows the two storage layouts side by side.)

Monitoring: two kinds

The guide never covers this, and the JD explicitly asks for "production monitoring," so know that it splits in two:

- **System monitoring** — the ordinary "is the service healthy" stuff: latency, throughput, error rate, GPU utilization, cost. Tools: **Prometheus + Grafana**.

- **Model monitoring** — the ML-specific part: quality degradation, **drift** (live data drifting away from what the model expects), bad or hallucinated outputs, and gathering feedback. Tools: **Evidently**, **Arize**, **WhyLabs**; for LLM/RAG specifically, **LangSmith** or **Langfuse** for tracing every retrieval and generation.

> KEY IDEA
>
> Model monitoring is the one people forget, and it's where RAG systems quietly fail — usually because retrieval starts returning irrelevant chunks, not because the model broke. A single "uptime is green" dashboard tells you nothing about whether answers are still *good*.

The pieces the guide (and most beginners) skip

- **Security & access control** — API keys, authentication, not leaking one user's data to another (recall Pinecone *namespaces* / multi-tenancy from guide §6.3).
- **Evaluation harness** — a way to measure whether answers are actually good *before and after* you change anything (RAGAS from guide §4.5, run as a repeatable job, not a one-off).
- **Cost controls** — token budgets, caching, the reranker trick (retrieve 25 cheaply, pass 4 to the expensive model) from guide §4.5.

Part XIV — The Full-Stack & Infrastructure Half of the JD

Here is the honest, important part. The guide is an *AI* guide. The job description is for a **Full Stack** AI/ML developer — and roughly half of it is web development and infrastructure the guide doesn't touch at all. This section maps that gap so you know what you're walking into.

> KEY IDEA
>
> The JD title is "AI/ML **Full Stack** Developer." That word is load-bearing. About 40% of the JD (LLMs, RAG, agents, vector DBs, the AI ecosystem tools) the guide covers cold. Another ~30% (MLOps, CI/CD, Docker, Kubernetes, cloud) this companion covers *conceptually*. The remaining ~30% — building actual frontend and backend web applications, plus databases and distributed systems — is a genuine third pillar neither document teaches, and it's real, hands-on software engineering.

Backend — the layer that serves your AI

The JD asks for **Node.js, Django, or FastAPI**. For an AI role, **FastAPI** is the one to prioritize:

- **FastAPI** — a modern, async Python web framework. It's the default for putting an ML model or RAG chain behind a REST API because it's Python (so it sits next to your model code), it's fast, and it uses **Pydantic** for request/response validation — the *same* Pydantic you already saw in the guide's `with_structured_output` (§3.4). If you understood that, you're halfway to FastAPI.
- **Django** — a "batteries-included" Python framework with a built-in ORM (database layer), admin panel, and auth. Heavier; better for full web apps than for a thin model API.
- **Node.js** (usually with Express) — the JavaScript backend option, common when the team is JS-first.

> **KEY IDEA**
>
> The orchestration layer from Part XIII is, concretely, *a FastAPI app*. A minimal production RAG backend is: a FastAPI endpoint that receives a question, runs the LangChain/LlamaIndex RAG chain from the guide, and returns the answer as JSON. That single sentence connects everything you've learned to a deployable service.

Frontend — what the user actually sees

The JD asks for **React.js, Next.js, or Angular**:

- **React** — the dominant UI library. Components, state, hooks. This is the one to learn first.
- **Next.js** — a React *framework* adding routing, server-side rendering, and API routes. Common for AI product frontends because it can host both the UI and lightweight backend routes.
- **Angular** — the older, more opinionated full framework; less common in new AI startups but still widespread in enterprise.

The guide's Project 1 even nudges you here: it suggests wrapping your RAG bot in **Streamlit** for a quick demo UI. Streamlit isn't React, but it makes an AI project *demoable in an interview* without frontend skills — a pragmatic bridge while you learn React.

Databases — SQL vs NoSQL

The JD asks for **PostgreSQL, MongoDB, or MySQL**:

- **PostgreSQL** and **MySQL** — *relational* (SQL) databases: structured tables, rows, columns, joins. Postgres is the more powerful/modern choice (and, via **pgvector**, doubles as a vector DB — Part XIII).

- **MongoDB** — a *document* (NoSQL) database: stores flexible JSON-like documents, no fixed schema.

> INTERVIEW QUESTION
>
> **Q:** When would you choose a relational database over a document database? **A:** Relational (Postgres/MySQL) when the data is structured and relationships matter — users, orders, transactions — and you want strong consistency and rich queries with joins. Document (MongoDB) when the shape of the data varies or evolves quickly, or you're storing loosely-structured records and want to move fast without migrations. For an AI app I'd often use Postgres for the core relational data *and* pgvector for embeddings, keeping it in one system rather than adding a separate vector DB before scale demands it.

Distributed systems, APIs, microservices

The JD asks for understanding of "software architecture, APIs, distributed systems, and scalable application design." The one-line versions:

- **REST API** — the standard way services talk over HTTP: predictable URLs, JSON in and out. Your FastAPI endpoint is a REST API.
- **Microservices** — splitting a big app into small, independently deployable services (e.g. the model service, the retrieval service, the frontend) instead of one monolith. Kubernetes (Part XI) is what runs them.
- **Distributed systems** — the general challenge of many machines working together reliably: load balancing, handling failures, keeping data consistent. This is the *why* behind Kubernetes and managed platforms.

> COMMON MISTAKE
>
> Under-preparing the full-stack half because the AI half is more exciting. On a "full-stack AI" role, interviewers will absolutely probe "how would you build the frontend / the API / the database schema for this feature," not just "explain RAG." A brilliant RAG answer plus a blank stare at "how would you expose this as an API and show it in a UI" is a weaker candidate than someone solid on both.

Part XV — Diffusion Models & Computer Vision (brief gap fill)

The JD mentions "Computer Vision, Generative AI, LLMs, or **Diffusion Models**." The guide covers image *classification* (Project 4) but not generative image models. The concepts, briefly:

Diffusion models (Stable Diffusion, DALL·E, Midjourney's class of model) generate images. The core idea:

- **Forward process:** take a real image and gradually add random noise over many steps until it's pure static.
- **Reverse process:** train a neural network (a **U-Net**) to *undo* that — to remove a little noise at each step. Run the reverse process starting from pure noise, and it "denoises" into a coherent image.
- **Text conditioning:** to get "a cat astronaut," the denoising is guided by a text embedding (often from a model like CLIP), so each denoising step is nudged toward the prompt.
- **Latent diffusion** (what makes Stable Diffusion efficient): do all of this in a compressed *latent* space rather than on full-resolution pixels, which is far cheaper.

> ANALOGY
>
> A diffusion model is a sculptor working in reverse. Instead of a finished statue slowly being buried in sand (noise), the model learns to brush sand away, grain by grain, until a statue emerges from what started as a featureless pile — and the text prompt tells it *which* statue to uncover.
>
> **KEY IDEA** Note the parallel to backprop from the guide: diffusion is still "train a network to minimize a loss with gradient descent." The novelty is *what* it's trained to predict — the noise to remove at each step. If you understand the training loop, diffusion is a new objective, not a new universe.

For **computer vision** more broadly: the guide's image classifier uses a plain feedforward net for simplicity, but real CV uses **CNNs** (convolutional neural networks) and increasingly **Vision Transformers**. Transfer learning (guide §8.7) — freezing a pretrained model like ResNet and training a small new head — is how nearly all applied CV is done.

Part XVI — New Interview Questions (deployment, MLOps, production)

These extend the guide's §10.1 into the topics it didn't cover. Practice them out loud.

31. How do you deploy a model to production? Put it behind an endpoint — a URL that takes an input and returns a prediction. Concretely: package it in a Docker container, then either run that on Kubernetes yourself, use a managed platform like SageMaker/ Vertex AI that provisions the GPU endpoint and scaling for you, or — if you're using a third-party model — just call a hosted API and deploy no model at all. The choice is driven by whether you need your own model and whether data must stay in-house.

32. What's the difference between an ML pipeline and a deployment pipeline? An ML pipeline is the ML work that produces a model: collect → preprocess → train → evaluate. A deployment (CI/CD) pipeline is the automation that ships a finished model: build → test → deploy → monitor. Deployment is usually the *last stage* of the ML pipeline or a separate pipeline triggered once a model is ready — they connect but aren't the same thing.

33. Walk me through the ML lifecycle. Six stages with a feedback loop: data collection → preprocessing/feature engineering → training → evaluation (the gate) → deployment → monitoring. Monitoring detects drift or decay and triggers retraining, looping back to the top. It's a cycle, not a line — a model is maintained, not built once.

34. What is Docker and why use it for ML? A container packages the model plus its exact environment — code, library versions, dependencies — into a portable image that runs identically everywhere. It solves "works on my machine, breaks in production," and it's the unit that Kubernetes, SageMaker, and CI/CD all operate on.

35. What does Kubernetes do that Docker doesn't? Docker packages and runs *one* container. Kubernetes *orchestrates many*: it keeps N copies running, restarts crashed ones, load-balances traffic, autoscales up and down, and spreads containers across machines. Docker is the shipping container; Kubernetes is the port.

36. What is MLflow for? Experiment tracking and a model registry. It logs each training run's parameters and metrics so you can compare them, and versions your trained models with stages like staging/production. It's the source of truth for *which* model is live and how it performed — essential for reproducibility.

37. What is data drift and how do you catch it? Drift is when live production data diverges from the data the model was trained on, silently degrading accuracy over time. You catch it with model monitoring — tools like Evidently or Arize compare the distribution of incoming data (and predictions) against a baseline and alert when they diverge, which is your signal to retrain.

38. How do you monitor an LLM/RAG system in production? Two layers. System monitoring — latency, errors, cost, throughput (Prometheus/Grafana). Model monitoring — answer quality, hallucination rate, and crucially *retrieval quality*, using RAGAS-style metrics and LLM tracing tools like LangSmith or Langfuse. RAG usually degrades because retrieval starts missing, not because the model breaks, so watching retrieval is non-negotiable.

39. What's CI/CD in an ML context? The automation that takes a code or model change and safely ships it: build the container, run tests and *evaluation*, deploy. In ML it often extends to CT — continuous training — automatically retraining on fresh data. It's what makes releases repeatable instead of a risky manual push. Tools: GitHub Actions, GitLab CI, Jenkins.

40. How would you expose a RAG chain as a production service? Wrap it in a FastAPI endpoint that receives a question, runs the LangChain/LlamaIndex RAG chain, and returns JSON with the answer and citations. Containerize that with Docker, deploy it (managed platform or Kubernetes), keep an offline ingestion job populating the vector DB, and add monitoring on latency, cost, and retrieval quality. That's the whole architecture in one breath.

Part XVII — Honest JD Coverage Map & Study Plan

Here's a candid assessment of where the guide + this companion put you against the actual job description.

What you've got cold (guide covers deeply)

- What an LLM is and its limitations; tokens, context windows, temperature
- Embeddings and semantic search
- RAG end to end — indexing, querying, chunking, hybrid search, reranking, evaluation, debugging
- LangChain (LCEL, tools, memory, structured output) and LangGraph (state, nodes, edges, agents, multi-agent)
- Vector databases — Chroma, Pinecone, Weaviate — and how to choose
- LlamaIndex and when to use it vs LangChain
- The training loop, backprop, overfitting, transfer learning; PyTorch vs Keras
- **This maps directly onto the JD's entire "Preferred Qualifications" section and much of "Generative AI / LLMs / NLP."**

What you now understand conceptually (this companion covers)

- Model deployment paths and what "deploy" means
- The MLOps toolchain — Docker, Kubernetes, SageMaker, Vertex AI, MLflow, Kubeflow, CI/CD
- ML pipeline vs deployment pipeline; the full lifecycle and feedback loop
- Production RAG architecture; monitoring (system + model); pgvector storage
- **This maps onto the JD's "MLOps, CI/CD, model serving, infrastructure" requirements — but conceptually, which is not the same as hands-on.**

The genuine gaps (neither document makes you job-ready alone)

- **Hands-on full-stack web development** — actually building React/Next frontends and FastAPI/Django/Node backends, wiring them to databases. This is ~30% of the role and can only be closed by *building*.

- **Hands-on cloud/MLOps** — you understand SageMaker/Docker/Kubernetes conceptually; the JD wants you to have *used* them. Reading about Kubernetes and deploying to it are different skills.
- **Production experience at 3–6 years depth** — the war stories interviewers actually want ("I had this problem, here's what I did") come from shipping, not reading.
- **CV/Diffusion depth** — you have the concepts (Part XV); the JD lists them as needed experience.
- **Math foundations** — the guide touches linear algebra and calculus lightly; the JD lists them explicitly. Worth a refresher on the *intuition* (what a gradient is, what matrix multiply does) even if not the proofs.

The single highest-leverage move

Build the guide's **Project 5 capstone and deploy it** — it's practically a checklist of this JD. It uses RAG, LangGraph agents, LlamaIndex, a vector DB, tools, memory, human-in-the-loop, citations, *and* a Streamlit UI, and the guide explicitly says "deploy it, put the link on your CV." Doing that end-to-end — then putting a real FastAPI + React front on it, containerizing it with Docker, and deploying it to one cloud — converts a huge fraction of this JD from "I've read about it" to "I built and shipped that," which is the answer interviewers are actually listening for.

> **FINAL, HONEST NOTE**
>
> This JD is broad — it asks one person to be a strong ML engineer *and* a strong MLOps engineer *and* a strong full-stack web developer. That's a wide ask, and job descriptions always list the ideal, not the hard minimum. Your foundation on the AI/RAG/agents side is genuinely strong — that's the part these roles probe hardest and where you can clearly shine. Go in confident about that, be honest about where your depth is conceptual vs hands-on (interviewers respect "I've built X, I understand Y conceptually and am closing that gap" far more than bluffing), and close as much of the build gap as you can before the interview. Confidence plus one deployed end-to-end project is a strong position to walk in with. You've done the hard conceptual work; the remaining step is turning some of it into things you've actually shipped.